

Leonardo Tomasela Leal
170291

Projeto de um Processador RISC-V

São José dos Campos - Brasil
Abril de 2026

Leonardo Tomasela Leal
170291

Projeto de um Processador RISC-V

Relatório apresentado à Universidade Federal de São Paulo como parte dos requisitos para aprovação na disciplina de Laboratório de Sistemas Computacionais: Arquitetura e Organização de Computadores.

Docente: Prof. Dr. Sérgio Ronaldo Barros dos Santos

Universidade Federal de São Paulo - UNIFESP

Instituto de Ciência e Tecnologia - Campus São José dos Campos

São José dos Campos - Brasil

Abril de 2026

Resumo

Este relatório descreve a implementação em SystemVerilog dos componentes fundamentais da unidade de processamento de um processador RISC-V de 32 bits, seguindo o subconjunto RV32IM e operando em ciclo único. Foram projetados, simulados e validados individualmente: a Unidade Lógica e Aritmética (ULA) com suporte a 24 operações, incluindo a extensão de multiplicação e divisão (RV32M); o banco de registradores com 32 registradores de 32 bits (escrita síncrona e leitura combinacional); a memória de instruções (ROM) inicializada a partir de arquivo texto; a memória de dados (RAM) síncrona com lógica de *byte enable* para acessos a byte, meia palavra e palavra, tratando corretamente a extensão de sinal (lb, lh) e de zero (lbu, lhu); o gerador de imediatos, que extrai e estende os valores imediatos para os diferentes tipos de instrução (I, S, B, U, J); a unidade de branch, responsável por calcular o próximo endereço do Contador de Programa para desvios condicionais e saltos (`jal`, `jalr`); e o Contador de Programa (PC), descrito como bloco síncrono com reset, a ser integrado no topo do processador. Além disso, realizou-se a integração entre o banco de registradores e a ULA, executando um programa de teste que compreendeu somas com imediato e uma divisão. Todos os módulos foram simulados no ambiente ModelSim, e os resultados das simulações são apresentados e analisados por meio de formas de onda (waveforms), comprovando a conformidade com a especificação RISC-V. O trabalho estabelece a base operacional para a futura integração do processador completo.

Palavras-chave: RISC-V. Unidade Lógica e Aritmética. Banco de registradores. Memória de instruções. Memória de dados. Gerador de imediatos. Unidade de branch. Contador de Programa. SystemVerilog. ModelSim.

Lista de abreviaturas e siglas

ALU	Arithmetic Logic Unit (Unidade Lógica e Aritmética)
AUIPC	Add Upper Immediate to Program Counter
BR	Banco de Registradores (Register File)
CISC	Complex Instruction Set Computer
CPLD	Complex Programmable Logic Device
DIV	Division
EA	Endereço Efetivo
FPGA	Field-Programmable Gate Array
HDL	Hardware Description Language
ISA	Instruction Set Architecture
JAL	Jump and Link
JALR	Jump and Link Register
LUI	Load Upper Immediate
MUL	Multiplication
PC	Program Counter (Contador de Programa)
RAM	Random Access Memory
REM	Remainder
RISC	Reduced Instruction Set Computer
ROM	Read-Only Memory
RV32I	RISC-V 32-bit Integer Base Instruction Set
RV32IM	RV32I + Integer Multiplication and Division Extension
SRA	Shift Right Arithmetic
SRL	Shift Right Logical

SV	SystemVerilog
ULA	Unidade Lógica e Aritmética
UVM	Universal Verification Methodology

Lista de ilustrações

Figura 1 – Arquitetura de Von Neumann e de Harvard	16
Figura 2 – Endereçamento por registrador	17
Figura 3 – Endereçamento por imediato	18
Figura 4 – Endereçamento por deslocamento	18
Figura 5 – Endereçamento PC-relativo	19
Figura 6 – Field-Programmable Gate Array	25
Figura 7 – Caminho de Dados Completo	31
Figura 8 – Caminho de Dados das Instruções do Tipo R	32
Figura 9 – Caminho de Dados das Instruções Aritméticas e Lógicas com Imediato	33
Figura 10 – Caminho de Dados das Instruções de Carregamento	34
Figura 11 – Caminho de Dados das Instruções de Armazenamento	35
Figura 12 – Caminho de Dados das Instruções de Desvio Condicional	36
Figura 13 – Caminho de Dados da Instrução de Salto e Ligação	37
Figura 14 – Caminho de Dados da Instrução de Salto Indireto	38
Figura 15 – Caminho de Dados de Carregamento de Imediato Superior	39
Figura 16 – Caminho de Dados da Adição de Imediato Superior ao PC	40
Figura 17 – Waveform das instruções do tipo R aritméticas (ADD, SUB, MUL, MULH, MULHSU, MULHU, DIV, DIVU, REM, REMU)	46
Figura 18 – Waveform das instruções de deslocamento (SLL, SRL, SRA)	47
Figura 19 – Instruções do Tipo R Lógicas e Comparações	48
Figura 20 – Waveform das instruções do tipo I (ADDI, XORI, ORI, ANDI, SLLI, SRLI, SRAI, SLTI, SLTIU).	49
Figura 21 – Waveform do cálculo de endereço para LOAD/STORE.	50
Figura 22 – Waveform das instruções de desvio condicional (BEQ, BNE, BLT, BGE, BLTU, BGEU).	51
Figura 23 – Waveform das instruções JAL, JALR e AUIPC.	52
Figura 24 – Waveform do banco de registradores 1	55
Figura 25 – Waveform do banco de registradores 2	55
Figura 26 – Waveform da simulação integrada Banco de Registradores e ULA.	58
Figura 27 – Waveform da memória de instruções com RESET ativo (RESET = 0).	61
Figura 28 – Waveform da memória de instruções com RESET inativo (RESET = 1).	62
Figura 29 – Waveform da simulação da memória de dados.	66
Figura 30 – Waveform da simulação da unidade de branch.	69
Figura 31 – Waveform da simulação do gerador de imediatos.	73

Lista de tabelas

Tabela 1	–	Tipos de instruções (formatos R, I, S, B, U)	27
Tabela 2	–	Formato da instrução <code>jal</code> (Tipo J)	27
Tabela 3	–	Instruções do tipo R	28
Tabela 4	–	Instruções do tipo I	29
Tabela 5	–	Instruções do tipo S	29
Tabela 6	–	Instruções do tipo B	30
Tabela 7	–	Instruções do tipo U	30
Tabela 8	–	Instrução do tipo J	30
Tabela 9	–	Programa de exemplo armazenado na memória de instruções.	60

Sumário

1	INTRODUÇÃO	10
2	OBJETIVOS	12
2.1	Geral	12
2.2	Específicos	12
3	FUNDAMENTAÇÃO TEÓRICA	14
3.1	Computador	14
3.2	Organização de Computadores	14
3.3	Arquitetura de Computadores	14
3.3.1	CISC vs RISC	14
3.3.1.1	Arquitetura CISC (Complex Instruction Set Computer)	15
3.3.1.2	Arquitetura RISC (Reduced Instruction Set Computer)	15
3.3.2	Arquitetura de Von Neumann vs. Arquitetura de Harvard	15
3.3.2.1	Arquitetura de Von Neumann	16
3.3.2.2	Arquitetura de Harvard	16
3.3.3	Modos de Endereçamento	17
3.3.3.1	Endereçamento por registrador	17
3.3.3.2	Endereçamento por imediato	18
3.3.3.3	Endereçamento por deslocamento	18
3.3.3.4	Endereçamento PC-relativo	18
3.3.4	Arquitetura RISC-V	19
3.3.4.1	Origem	19
3.3.4.2	Características	19
3.3.4.3	Modos de Endereçamento	20
3.3.5	Justificativa da Escolha	20
3.4	Linguagens de Descrição de Hardware (HDLs)	20
3.4.1	A Linguagem Verilog	21
3.4.2	A Linguagem SystemVerilog	23
3.5	FPGA (Field-Programmable Gate Array)	24
4	DESENVOLVIMENTO	26
4.1	Características Gerais do Processador	26
4.2	Desenvolvimento da Arquitetura	26
4.2.1	Instruções do Tipo R	27
4.2.2	Instruções do Tipo I	28

4.2.3	Instruções do Tipo S	29
4.2.4	Instruções do Tipo B	29
4.2.5	Instruções do Tipo U	30
4.2.6	Instruções do Tipo J	30
4.3	Desenvolvimento da Organização	30
4.3.1	Caminho de Dados Completo	31
4.3.2	Caminho de Dados das Instruções do Tipo R	32
4.3.3	Caminho de Dados das Instruções Aritméticas e Lógicas com Imediato (<i>addi, xori, ori, andi, slli, srli, srai, slti, sltiu</i>)	32
4.3.4	Caminho de Dados das Instruções de Carregamento (<i>lb, lh, lw, lbu, lhu</i>)	33
4.3.5	Caminho de Dados das Instruções de Armazenamento (<i>sb, sh, sw</i>)	34
4.3.6	Caminho de Dados das Instruções de Desvio Condicional (<i>beq, bne, blt, bge, bltu, bgeu</i>)	35
4.3.7	Caminho de Dados da Instrução de Salto e Ligação (<i>jal</i>)	36
4.3.8	Caminho de Dados da Instrução de Salto Indireto (<i>jalr</i>)	37
4.3.9	Caminho de Dados de Carregamento de Imediato Superior (<i>lui</i>)	38
4.3.10	Caminho de Dados da Adição de Imediato Superior ao PC (<i>auipc</i>)	39
4.4	Definições Globais e Package <i>data_structures</i>	40
4.5	Desenvolvimento da Unidade Lógica e Aritmética (ALU)	41
4.5.1	Interface da ALU	41
4.5.2	Decodificação da Operação Mapeamento para <i>aluop_t</i>	42
4.5.3	Geração dos Operandos	43
4.5.4	Implementação das Operações	43
4.5.5	Desenvolvimento dos Testbenches	44
4.5.5.1	Estrutura do Testbench	44
4.5.5.2	Organização dos Estímulos	45
4.5.6	Resultados da Simulação da ALU	46
4.5.6.1	Instruções do tipo R	46
4.5.6.1.1	Aritméticas	46
4.5.6.1.2	Shift	47
4.5.6.1.3	Lógicas	48
4.5.6.2	Instruções do Tipo I (Aritmética com Imediato)	49
4.5.6.3	Instruções de Load e Store	50
4.5.6.4	Instruções de Desvio Condicional	51
4.5.6.5	Instruções de Salto (JAL, JALR) e AUIPC	52
4.6	Banco de Registradores	53
4.6.1	Implementação do Módulo <i>regfile</i>	53
4.6.2	Testbench do Banco de Registradores	54
4.6.3	Resultados da Simulação	55

4.7	Integração Banco de Registradores e ULA	56
4.7.1	Testbench de Integração	56
4.7.2	Resultados da Simulação Integrada	58
4.8	Memória de Instruções	59
4.8.1	Implementação do Módulo <code>inst_memory</code>	59
4.8.2	Arquivo de Inicialização <code>dados.txt</code>	60
4.8.3	Testbench da Memória de Instruções	60
4.8.4	Resultados da Simulação	61
4.9	Memória de Dados	63
4.9.1	Implementação do Módulo	63
4.9.2	Testbench da Memória de Dados	65
4.9.3	Resultados da Simulação	66
4.10	Unidade de Branch	67
4.10.1	Implementação do Módulo <code>branch_unit</code>	67
4.10.2	Testbench da Unidade de Branch	68
4.10.3	Resultados da Simulação	69
4.11	Contador de Programa	70
4.12	Gerador de Imediatos	71
4.12.1	Implementação do Módulo <code>immediate_generator</code>	71
4.12.2	Testbench do Gerador de Imediatos	72
4.12.3	Resultados da Simulação	73
5	CONSIDERAÇÕES FINAIS	75
5.0.1	Dificuldades Encontradas	76
	REFERÊNCIAS	78

1 Introdução

"Os computadores levaram a uma terceira revolução para a civilização, com a revolução da informação tomando o seu lugar ao lado da agricultura e revoluções industriais." (PATTERSON; HENNESSY, 2018, p. 41, tradução própria). A evolução dos sistemas computacionais está intrinsecamente ligada ao constante aprimoramento das suas arquiteturas e da sua organização física. O estudo prático do funcionamento lógico de um processador apresenta-se como uma etapa indispensável para a consolidação teórica dos conceitos abordados. Neste contexto, o presente relatório documenta o projeto de um processador digital baseado na arquitetura RISC-V de 32 bits, operando sob um modelo de execução de ciclo único (monociclo).

A motivação central para a adoção da norma RISC-V no desenvolvimento deste projeto reside na sua natureza de padrão de código aberto (open source) e na sua licença totalmente gratuita. Estas características posicionam o RISC-V como uma alternativa acadêmica e industrial altamente viável e moderna face a arquiteturas proprietárias e fechadas que dominam o mercado atual, tais como as famílias x86 e ARM. Para além da acessibilidade financeira e filosófica, o RISC-V destaca-se pela sua profunda modularidade. O seu conjunto de instruções foi estrategicamente desenhado para ser enxuto na base, sendo suportado por várias extensões normalizadas, o que o torna fácil de adaptar para diversos usos de processamento. Esta modularidade é o que viabiliza a inclusão, neste processador, de um bloco específico de multiplicação e divisão em hardware acoplado à base de inteiros (constituindo um subconjunto da variante RV32IM).

Por conseguinte, o objetivo central deste documento é expor, de forma rigorosa, as decisões arquiteturais e organizacionais tomadas durante a elaboração do hardware. O relatório encontra-se estruturado de modo a apresentar, numa primeira instância, os objetivos específicos do projeto, seguidos de uma fundamentação teórica sólida sobre os paradigmas computacionais, linguagens de descrição de hardware (Verilog e SystemVerilog) e FPGAs. Posteriormente, são detalhados o subconjunto de instruções implementado e os seus respectivos formatos, culminando na apresentação do desenvolvimento da organização estrutural, onde se discriminam os componentes do sistema e o desenho do caminho de dados (datapath). São descritas a implementação da Unidade Lógica e Aritmética (ULA), do banco de registradores, da integração entre eles, das memórias de instrução e dados, da unidade de branch (cálculo do próximo PC para desvios e saltos) e do Contador de Programa (PC), cuja lógica de atualização síncrona é apresentada. Finalmente, as simulações realizadas e os resultados obtidos são analisados, confirmando o funcionamento esperado de todos os componentes.

...

2 Objetivos

2.1 Geral

Implementar e validar, por meio da linguagem de descrição de hardware System-Verilog, os componentes fundamentais da unidade de processamento de um processador RISC-V de 32 bits, compreendendo a Unidade Lógica e Aritmética (ULA), o banco de registradores, as memórias de instrução e de dados, bem como a integração desses módulos, de modo a estabelecer a base operacional para a construção de um processador monociclo completo.

2.2 Específicos

Para a realização do trabalho, os seguintes objetivos específicos foram definidos e atendidos:

1. Implementar a Unidade Lógica e Aritmética (ULA) com suporte às operações da base inteira RV32I (adição, subtração, operações lógicas, deslocamentos e comparações) e à extensão de multiplicação e divisão RV32M, totalizando 24 operações.
2. Projetar o banco de registradores (Register File) com 32 registradores de 32 bits, respeitando a característica de **x0** permanentemente igual a zero e utilizando escrita síncrona e leitura combinacional.
3. Desenvolver a memória de instruções (ROM) capaz de carregar um programa a partir de um arquivo texto e fornecer a instrução correta para cada endereço solicitado.
4. Implementar a memória de dados (RAM) síncrona de 1024 palavras, com suporte a acessos a byte, meia palavra e palavra, incluindo a lógica de *byte enable* e a correta extensão de sinal ou de zero conforme o tipo de acesso (**lb**, **lbu**, **lh**, **lhu**, **lw**, **sb**, **sh**, **sw**).
5. Implementar o gerador de imediatos (Immediate Generator) para extrair e estender os valores imediatos dos formatos I, S, B, U e J, conforme a especificação RISC-V.
6. Desenvolver a unidade de branch para cálculo do próximo PC em desvios condicionais, saltos incondicionais (**jal**) e saltos indiretos (**jalr**), utilizando sinais de controle e o bit de condição da ALU.

7. Realizar a integração entre o banco de registradores e a ULA, verificando o fluxo completo de leitura de operandos, processamento e escrita de resultados por meio de um testbench conjunto.
8. Criar testbenches modulares para cada componente, simulando-os no ambiente ModelSim e documentando os resultados por meio de formas de onda (waveforms).

3 Fundamentação Teórica

Esta seção tem como objetivo estabelecer a base teórica e conceitual que fundamenta o desenvolvimento do projeto. Nela, são abordados os princípios fundamentais de arquitetura e organização de computadores, os paradigmas de memória e modos de endereçamento, bem como as principais características da arquitetura RISC-V, fornecendo o embasamento literário necessário para justificar as escolhas de projeto adotadas.

3.1 Computador

Define-se, em sua essência, como um sistema digital e eletrônico projetado para receber, armazenar, processar e transmitir informações de maneira automatizada, operando sob o controle estrito de um conjunto de instruções previamente armazenado.

3.2 Organização de Computadores

Refere-se à implementação física e de hardware responsável por concretizar as especificações da arquitetura proposta. O projeto organizacional envolve o arranjo das unidades operacionais e suas interconexões lógicas, englobando o desenho do caminho de dados (datapath), o dimensionamento dos sinais da Unidade de Controle e a hierarquia topológica do sistema de memórias.

3.3 Arquitetura de Computadores

Corresponde aos atributos lógicos de um sistema computacional que são diretamente visíveis ao programador de baixo nível ou ao desenvolvedor de compiladores. Ela engloba a definição formal do Conjunto de Instruções (ISA - Instruction Set Architecture), os tipos de dados matemáticos suportados, a quantidade de registradores de uso geral expostos e os métodos padronizados de acesso à memória. Em suma, a arquitetura estabelece as especificações funcionais que determinam o que o processador é capaz de realizar.

3.3.1 CISC vs RISC

A distinção entre os paradigmas CISC (Complex Instruction Set Computer) e RISC (Reduced Instruction Set Computer) é fundamental na classificação de processadores.

3.3.1.1 Arquitetura CISC (Complex Instruction Set Computer)

A arquitetura CISC caracteriza-se por um conjunto amplo e heterogêneo de instruções, com tamanhos variáveis e múltiplos modos de endereçamento. Neste modelo, uma única instrução pode realizar várias operações de baixo nível (como acessar a memória, processar o dado e guardar o resultado). Embora reduza o tamanho do código compilado, esta abordagem aumenta substancialmente a complexidade do hardware e da Unidade de Controle, resultando em ciclos de execução irregulares.

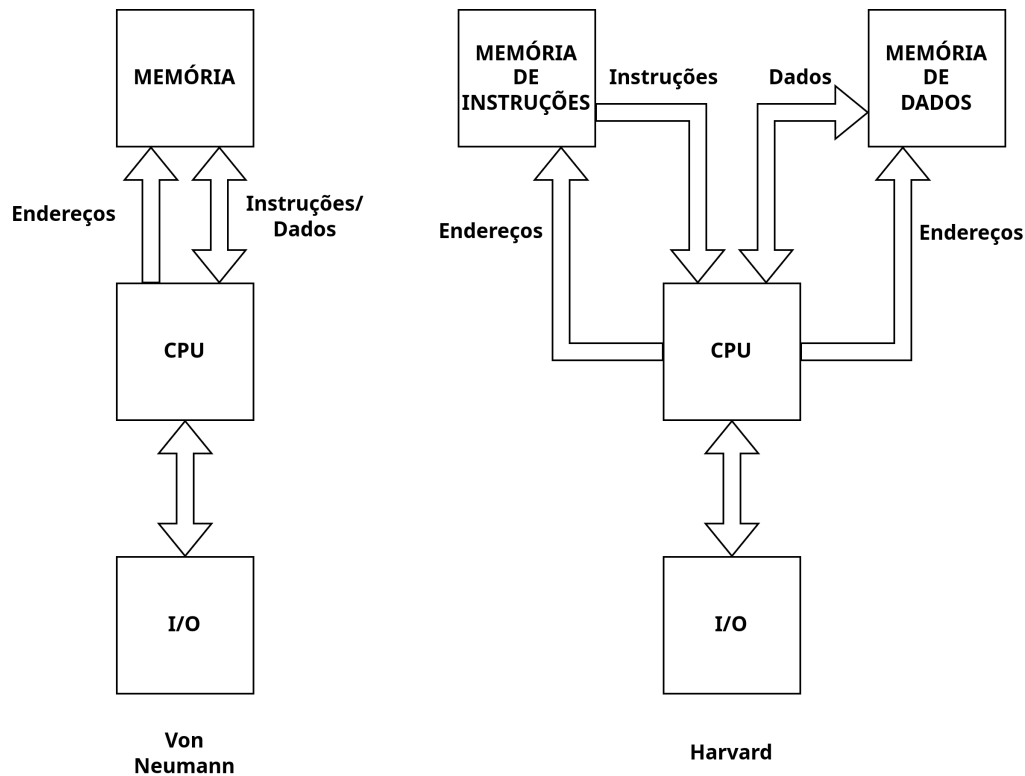
3.3.1.2 Arquitetura RISC (Reduced Instruction Set Computer)

A arquitetura RISC adota um conjunto reduzido de instruções simples, otimizadas e de formato fixo, o que agiliza drasticamente a decodificação. A sua característica estrutural central é a adoção rigorosa do modelo load-store, em que o processamento matemático e lógico ocorre exclusivamente entre os registradores internos. O acesso à memória principal é estritamente restrito a instruções dedicadas de leitura (load) e escrita (store). Esta filosofia simplifica o hardware, reduz os modos de endereçamento e garante a estabilidade previsível essencial para viabilizar a implementação de um caminho de dados de ciclo único.

3.3.2 Arquitetura de Von Neumann vs. Arquitetura de Harvard

A Figura 1 apresenta as diferenças entre as arquiteturas, que serão elaboradas nos subtópicos seguintes.

Figura 1 – Arquitetura de Von Neumann e de Harvard



Fonte: O Autor

3.3.2.1 Arquitetura de Von Neumann

Fundamentada no conceito clássico de programa armazenado, este modelo caracteriza-se por manter a memória de dados e a memória de instruções estruturadas de forma unificada. Consequentemente, o sistema dispõe de um barramento único para o tráfego de informações. Essa característica estrutural cria um afunilamento (conhecido como "Gargalo de Von Neumann"), que obriga o processador a executar as instruções em uma ordem estritamente sequencial, impossibilitando a busca de uma instrução e a manipulação de um dado no mesmo instante de tempo. É um padrão habitualmente associado aos processadores de instrução complexa (CISC).

3.3.2.2 Arquitetura de Harvard

Em contrapartida, este paradigma resolve o gargalo de concorrência ao adotar a separação física estrita entre a memória de dados e a memória de instruções. A utilização de barramentos de comunicação isolados possibilita que o processador efetue a busca da próxima instrução e a leitura ou escrita de um dado operacional de forma simultânea. Esta abordagem de acesso paralelo é inerente ao modelo RISC e torna-se um requisito vital para a implementação de um caminho de dados de ciclo único.

3.3.3 Modos de Endereçamento

Os modos de endereçamento consistem nos mecanismos lógicos e aritméticos pelos quais a arquitetura do hardware calcula os endereços efetivos dos operandos requisitados para processar uma instrução. Para esta seção utilizaremos a seguinte notação, baseada na utilizada por Stallings ([STALLINGS, 2017](#)) para explicar os modos de endereçamento:

1. **A** $\Rightarrow$ O conteúdo no campo de endereço dentro de uma instrução.
2. **PC** $\Rightarrow$ O conteúdo em uma instrução se refere ao registrador contador de programa.
3. **R** $\Rightarrow$ O conteúdo em uma instrução se refere a um registrador.
4. **(X)** $\Rightarrow$ O conteúdo do local de memória X ou do registrador X.
5. **EA** $\Rightarrow$ Endereço real do operando.

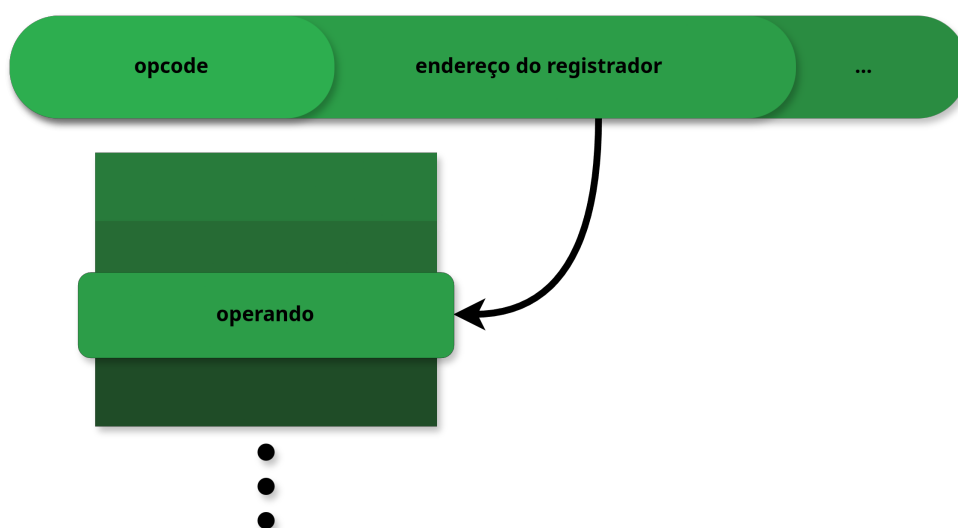
O escopo do projeto sustenta-se nos seguintes modos principais:

3.3.3.1 Endereçamento por registrador

A própria instrução decodificada faz uma referência direta ao endereço numérico de um registrador interno, que, por sua vez, armazena o operando necessário para a Unidade Lógica e Aritmética:

$$EA = R$$

Figura 2 – Endereçamento por registrador



Fonte: O Autor

3.3.3.2 Endereçamento por imediato

O valor numérico constante a ser utilizado na operação não é buscado no banco de registradores, pois já se encontra embutido fisicamente em um campo específico dos próprios bits da instrução:

$$EA = A$$

Figura 3 – Endereçamento por imediato



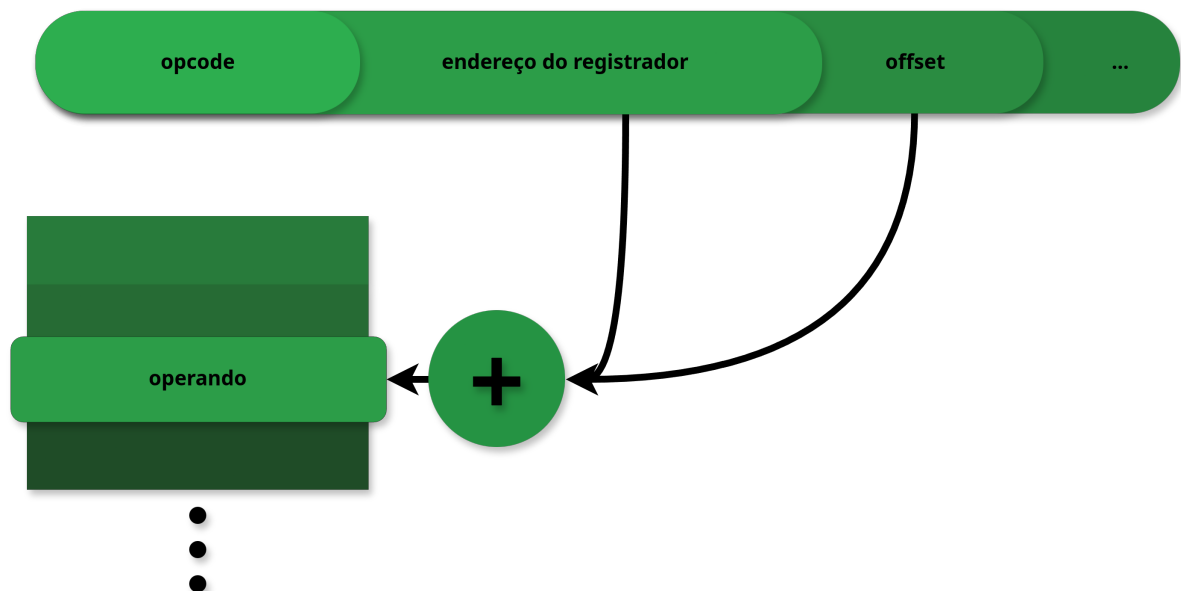
Fonte: O Autor

3.3.3.3 Endereçamento por deslocamento

O processo de acesso ao módulo de memória de dados é estabelecido somando-se um valor referencial de base (armazenado em um registrador) a um valor fixo de deslocamento (caracterizado como um offset imediato de 12 bits na ISA abordada):

$$EA = A + (R)$$

Figura 4 – Endereçamento por deslocamento



Fonte: O Autor

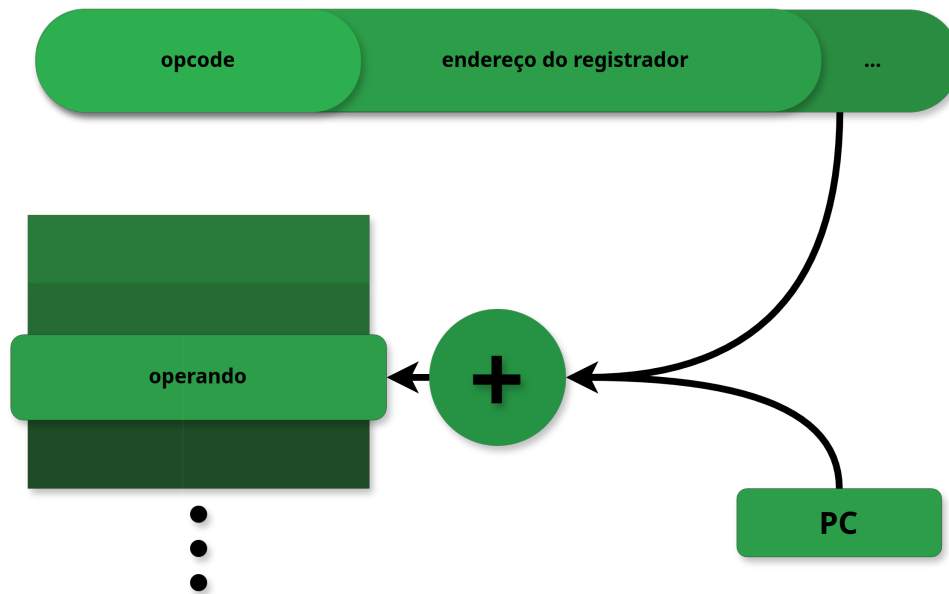
3.3.3.4 Endereçamento PC-relativo

Empregado no fluxo de controle de saltos e desvios, o endereço computado para o salto é obtido somando-se o valor atual armazenado no Contador de Programa (PC) a

um valor imediato extraído da instrução:

$$EA = A + (PC)$$

Figura 5 – Endereçamento PC-relativo



Fonte: O Autor

3.3.4 Arquitetura RISC-V

3.3.4.1 Origem

O RISC-V foi concebido e estruturado na Universidade da Califórnia em Berkeley (UC Berkeley) ao longo do verão do ano de 2010. O projeto foi idealizado e gerido por uma equipe de acadêmicos e pesquisadores englobando pioneiros da computação como Krste Asanovi e David Patterson. O desenvolvimento se originou da necessidade intrínseca da equipe por um processador de arquitetura moderna e de licença livre (gratuita), passível de ser implementado sem restrições em aulas práticas e pesquisas.

3.3.4.2 Características

O RISC-V é uma arquitetura estabelecida sobre os preceitos do modelo RISC (Reduced Instruction Set Computer). Diferenciando-se das arquiteturas complexas (CISC), ele é construído sobre o arranjo load-store (onde o acesso físico à memória restringe-se a instruções exclusivas de leitura e escrita), disponibiliza um largo conjunto de 32 registradores de uso geral e processa instruções com formato simples e de tamanho rigidamente fixado em 32 bits. O sistema utiliza um mapeamento de endereçamento com formato padrão orientado a byte de acordo com a ordem little endian ([RISC-V INTERNATIONAL, 2024](#)).

3.3.4.3 Modos de Endereçamento

Mantendo plena conformidade com sua filosofia de redução e simplificação computacional (cujo foco preza pela minimização da complexidade dos decodificadores), o conjunto de instruções do RISC-V opera unicamente sob o emprego dos modos de endereçamento por registrador (para manipulações puramente lógicas/matemáticas), por imediato (para interações com constantes numéricas), por deslocamento (recurso fundamental das instruções nativas de Load e Store) e de maneira PC-relativa (encarregado pelo direcionamento imperativo de saltos e desvios estruturados) ([RISC-V INTERNATIONAL, 2024](#)).

3.3.5 Justificativa da Escolha

A escolha definitiva da ISA RISC-V para conduzir o desenvolvimento lógico exposto neste relatório consolida-se, fundamentalmente, pelas garantias de seu padrão de código aberto (open source) atrelado à abstenção total de pagamentos e taxas de licenciamento comercial (gratuito). Essas propriedades proporcionam uma viabilidade sem precedentes e firmam a arquitetura como uma alternativa moderna, acadêmica e industrialmente proeminente frente aos competidores corporativos preestabelecidos, tais como os processadores das famílias x86 e ARM. Além dos benefícios econômicos e da disponibilidade, a base técnica justifica-se pela adoção de um design profundamente modular. O RISC-V disponibiliza um conjunto essencial de instruções rigorosamente simplificadas, sendo projetado de tal forma que é perfeitamente exequível adaptá-lo a variados cenários mediante o simples acoplamento de extensões. Essa prerrogativa arquitetural viabiliza e acomoda a inclusão direcionada do bloco de multiplicação em hardware abordada neste projeto.

3.4 Linguagens de Descrição de Hardware (HDLs)

O desenvolvimento de sistemas digitais complexos, como o processador RISC-V proposto, exige ferramentas de projeto robustas que vão além da tradicional entrada esquemática. As Linguagens de Descrição de Hardware (Hardware Description Languages - HDLs) surgem como uma alternativa poderosa, permitindo ao projetista descrever a estrutura e o comportamento de um sistema eletrônico por meio de um arquivo de texto, seguindo uma sintaxe e semântica bem definidas. Este arquivo de descrição serve como uma especificação executável do hardware e pode ser utilizado para gerar dados de programação para dispositivos lógicos programáveis ou para ser sintetizado em um circuito físico.

Diferentemente das linguagens de programação de software, que possuem um fluxo de execução sequencial de instruções (paradigma imperativo), as HDLs são inerentemente

concorrentes (ou paralelas). Em uma descrição de hardware, a execução das expressões e atribuições de valores ocorre de forma a emular a propagação de sinais por um circuito digital, onde múltiplas operações acontecem simultaneamente. Esta característica fundamental torna as HDLs a ferramenta ideal para a descrição formal de circuitos eletrônicos.

As principais vantagens da adoção de uma HDL no projeto de hardware incluem:

1. **Abstração e Hierarquia:** Permitem representar desde equações booleanas e tabelas verdade até operações matemáticas complexas, facilitando a criação de projetos hierárquicos onde um componente pode ser reutilizado como parte de outro.
2. **Síntese Lógica:** Uma descrição em HDL pode ser utilizada por uma ferramenta de síntese lógica para, em conjunto com uma biblioteca de componentes, gerar automaticamente a lista de interconexões (netlist) de um circuito otimizado.
3. **Simulação:** Permitem a simulação funcional do projeto para verificar seu comportamento antes mesmo de sua implementação física, garantindo a correteza do sistema em desenvolvimento.
4. **Documentação:** Servem como uma forma de documentação precisa e formal da estrutura e do funcionamento do hardware projetado.

Entre as diversas HDLs existentes, atualmente as mais conhecidas e utilizadas são o VHDL, que teve origem em um programa de pesquisa do Departamento de Defesa dos EUA, e o Verilog, que será a linguagem de escolha para a implementação do processador deste projeto.

3.4.1 A Linguagem Verilog

A linguagem Verilog foi desenvolvida para a descrição de circuitos eletrônicos digitais, sendo amplamente utilizada na documentação, simulação e síntese automática de sistemas digitais, especialmente em ambientes acadêmicos e industriais. Sua sintaxe possui semelhanças com a linguagem de programação C, o que facilita seu aprendizado para programadores familiarizados com este paradigma.

Um dos pontos mais importantes para a correta compreensão do Verilog é a clara distinção entre a modelagem de hardware e a modelagem de software:

1. **Modelagem de Software (linguagens como C, Python):** Assume-se um processador como unidade de execução. O programador escreve um código que será executado de forma sequencial pelo processador, passo a passo ao longo do tempo. Há uma exploração temporal da execução, onde um único somador pode ser reutilizado em múltiplas iterações de um laço de repetição.

2. **Modelagem de Hardware (Verilog, VHDL):** Modelam-se os próprios circuitos digitais. O projetista explora a execução não apenas no tempo, mas também no espaço, podendo replicar componentes (como somadores) para que operações sejam realizadas simultaneamente. A simultaneidade e o paralelismo são características intrínsecas da descrição.

Para a descrição de sistemas em Verilog, o código é organizado em módulos, que representam blocos de hardware com entradas e saídas definidas. Estes módulos podem ser instanciados e interconectados para formar sistemas de maior complexidade.

Dentro de um módulo, três estilos principais podem ser empregados para descrever seu funcionamento:

1. **Descrição Estrutural:** Descreve a interconexão entre componentes mais simples (sub-módulos ou portas lógicas), sendo análoga a uma lista de componentes e suas conexões.
2. **Descrição Comportamental:** Descreve o funcionamento desejado do circuito, especificando como as saídas devem reagir a mudanças nas entradas, sem necessariamente detalhar sua estrutura interna. Utiliza estruturas como `always` e `if-else`.
3. **Descrição de Fluxo de Dados (Dataflow):** Descreve o circuito por meio de equações que expressam a relação entre suas entradas e saídas, utilizando operadores lógicos e aritméticos em conjunto com a palavra-chave `assign`.

A seguir segue um exemplo de um mux simples de 4 bits na linguagem Verilog:

```
1  module mux41_verilog (
2      input [1:0] sel,
3      input a, b, c, d,
4      input clk, rst,
5      output reg y
6  );
7
8      // Sinal intermediario precisa ser reg porque recebe atribuicao no always
9      reg y_comb;
10
11     // Bloco Combinacional (Lista de sensibilidade generica)
12     always @(sel or a or b or c or d) begin
13         case (sel)
14             2'b00: y_comb = a;
15             2'b01: y_comb = b;
16             2'b10: y_comb = c;
17             2'b11: y_comb = d;
18             default: y_comb = 1'bx;
19         endcase
20     end
21
22     // Bloco Sequencial (Registrador de Saida)
23     always @(posedge clk or posedge rst) begin
```

```

24     if (rst)
25         y <= 1'b0;
26     else
27         y <= y_comb;
28     end
29
30 endmodule

```

Listing 3.1 – MUX 4:1 em Verilog Clássico

3.4.2 A Linguagem SystemVerilog

Para projetos de maior escala e complexidade, o SystemVerilog surge como uma extensão (superset) do Verilog tradicional. Ele incorpora características de linguagens de programação de alto nível, como orientação a objetos, e introduz um conjunto abrangente de ferramentas voltadas para a verificação funcional, incluindo geração de testes aleatórios restritos (constrained random testing), cobertura funcional (functional coverage) e metodologias de verificação como a Universal Verification Methodology (UVM). Embora este projeto utilize o Verilog padrão, o SystemVerilog representa o estado da arte para a verificação de circuitos integrados complexos.

No contexto deste trabalho, a linguagem Verilog é a escolhida para dar continuidade ao desenvolvimento do processador RISC-V, sendo a ponte entre a especificação arquitetural idealizada e a implementação real em hardware.

A seguir segue um exemplo do mesmo mux demonstrado anteriormente para a linguagem SystemVerilog:

```

1  module mux41_sv (
2      input  logic [1:0] sel,
3      input  logic      a, b, c, d,
4      input  logic      clk, rst,
5      output logic      y
6  );
7
8      // Nao precisa decidir entre reg/wire, usa-se apenas logic
9      logic y_comb;
10
11     // always_comb monta a lista de sensibilidade automaticamente
12     always_comb begin
13         unique case (sel)
14             2'b00: y_comb = a;
15             2'b01: y_comb = b;
16             2'b10: y_comb = c;
17             2'b11: y_comb = d;
18         endcase
19     end
20
21     // always_ff garante que o compilador valide se o circuito e sequencial
22     always_ff @(posedge clk or posedge rst) begin
23         if (rst) y <= '0; // Sintaxe '0 zera todos os bits automaticamente
24         else    y <= y_comb;
25     end

```


26
27 `endmodule`

Listing 3.2 – MUX 4:1 Simplificado em SystemVerilog

3.5 FPGA (Field-Programmable Gate Array)

Após a definição da arquitetura e a descrição do processador em uma HDL, a etapa seguinte consiste em materializar este projeto em um dispositivo físico capaz de executar o conjunto de instruções definido. Uma das plataformas mais adequadas para a prototipagem e implementação de sistemas digitais como este é o FPGA (Field-Programmable Gate Array).

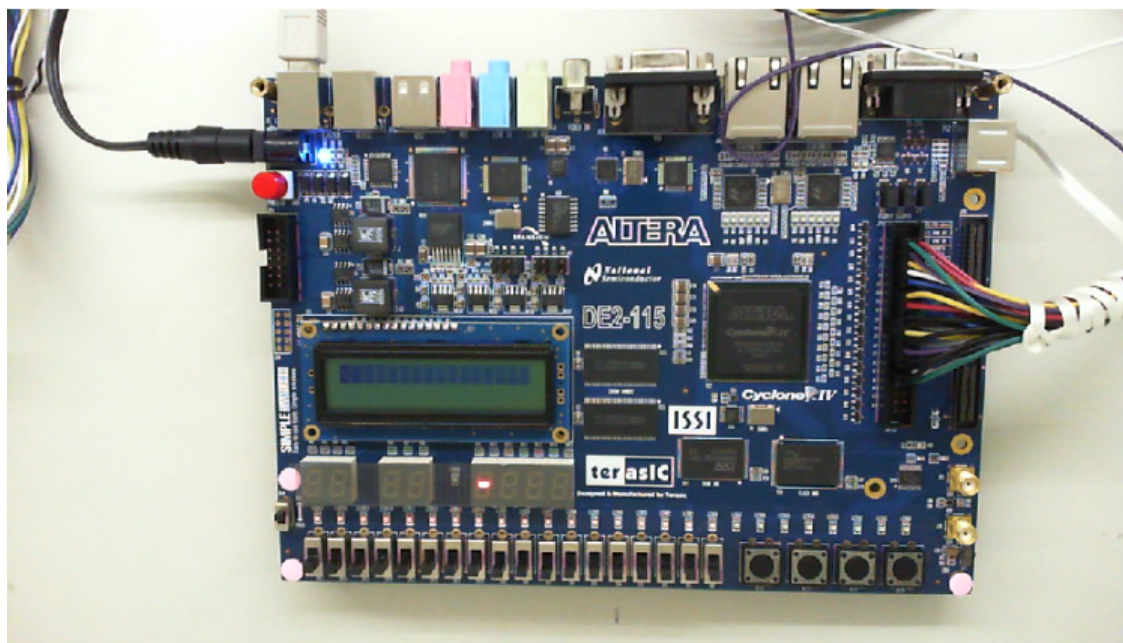
Um FPGA é um dispositivo semicondutor, cuja arquitetura se baseia em uma matriz regular de blocos lógicos programáveis (configuráveis para realizar funções lógicas simples ou complexas), circundada por uma matriz de interconexão programável que permite conectar estes blocos de forma flexível, e blocos de entrada e saída (I/O) para a comunicação com o mundo externo.

A principal característica que diferencia um FPGA de outros dispositivos lógicos programáveis (como CPLDs) é sua altíssima capacidade de integração, podendo conter desde alguns milhares até vários milhões de equivalentes a portas lógicas, e a natureza finamente granular de seus blocos lógicos. Enquanto um CPLD contém poucos blocos lógicos grandes, o FPGA contém uma vasta quantidade de blocos pequenos, oferecendo uma flexibilidade e densidade de integração muito superiores para circuitos complexos.

Os blocos de interconexão e roteamento são o ponto chave para a flexibilidade do FPGA. É através deles que o algoritmo de roteamento, parte das ferramentas de desenvolvimento do fabricante, define como as conexões entre os blocos lógicos serão estabelecidas para realizar o circuito projetado. Esta capacidade de reconfiguração é sua grande vantagem: o mesmo chip pode ser reprogramado inúmeras vezes para implementar funções de hardware completamente diferentes.

Relevância para o Projeto Para este trabalho, a implementação do processador RISC-V descrito em Verilog será sintetizada, mapeada e configurada em um FPGA. Esta abordagem permite verificar o funcionamento do processador em um cenário próximo ao real, executando programas de teste de forma acelerada. Assim como a adoção da arquitetura RISC-V garante a compatibilidade da descrição em HDL com um padrão aberto, a utilização de um FPGA garante a compatibilidade do projeto com uma plataforma de hardware flexível e reconfigurável, amplamente difundida tanto no meio acadêmico quanto no industrial para a prototipagem de sistemas digitais e sistemas embarcados.

Figura 6 – Field-Programmable Gate Array



Fonte: O Autor

4 Desenvolvimento

Neste capítulo, detalha-se o processo de concepção do processador, abrangendo desde as definições macroestruturais até a especificação técnica das instruções e a lógica de organização do hardware.

4.1 Características Gerais do Processador

O processador desenvolvido é um núcleo de processamento digital baseado na arquitetura RISC-V, implementando um subconjunto do conjunto de instruções RV32IM. As principais diretrizes técnicas que definem o sistema são:

1. **Largura de Dados:** 32 bits, operando com palavras (words) de quatro bytes.
2. **Modelo de Execução:** Monociclo (Ciclo Único), onde cada instrução é buscada, decodificada e executada em um único pulso de clock.
3. **Conjunto de Instruções (ISA):** Base inteira (I) com a extensão de multiplicação e divisão em hardware (M). Não está planejada a implementação das instruções *ebreak* e *ecall*, entretanto, caso necessário, isso pode mudar.
4. **Ordenação de Memória:** Padrão Little-endian, com endereçamento orientado a byte.
5. **Banco de Registradores:** Composto por 32 registradores de uso geral (x0 a x31), cada um com 32 bits de largura.
6. **Arquitetura de Memória:** Modelo Harvard, com a separação física entre a memória de instruções e a memória de dados para viabilizar o fluxo de dados em ciclo único.

4.2 Desenvolvimento da Arquitetura

A arquitetura RISC-V organiza suas instruções em formatos fixos de 32 bits para otimizar a decodificação em hardware ([RISC-V INTERNATIONAL, 2024](#)). Para este projeto, as instruções foram agrupadas em cinco categorias principais:

Tabela 1 – Tipos de instruções (formatos R, I, S, B, U)

31	25	24	20	19	15	14	12	11	7	6	0	
funct7		rs2		rs1		funct3		rd		opcode		Tipo R
imm[11:0]				rs1		funct3		rd		opcode		Tipo I
imm[11:5]		rs2		rs1		funct3		imm[4:0]		opcode		Tipo S
imm[11:5]		rs2		rs1		funct3		imm[4:0]		opcode		Tipo B
imm[31:12]										opcode		Tipo U

Fonte: O Autor

Tabela 2 – Formato da instrução jal (Tipo J)

31	3021	20	1912	117	60
imm[20]	imm[10:1]	imm[11]	imm[19:12]	rd	opcode
imediato = {imm[20], imm[10:1], imm[11], imm[19:12], 1'b0}					

Fonte: O Autor

4.2.1 Instruções do Tipo R

As instruções do Tipo R são projetadas para a execução de operações aritméticas e lógicas nas quais todos os operandos envolvidos residem internamente no banco de registradores. Consequentemente, esta categoria utiliza de forma exclusiva o modo de endereçamento por registrador, lendo os dados de duas fontes e gravando o resultado em um destino. Como exemplos práticos desta categoria implementados no projeto, destacam-se as operações lógicas e aritméticas fundamentais (como soma, subtração, operações lógicas and, or e xor), os deslocamentos de bits, as comparações de magnitude e, de modo integrado, as operações atreladas à Extensão M de hardware, compreendendo as funções de multiplicação e divisão em nível de silício.

Tabela 3 – Instruções do tipo R

Mnemônico	funct3	funct7	Opcode	Descrição	Notas
add	0x0	0x00	0110011	$rd = rs1 + rs2$	
sub	0x0	0x20	0110011	$rd = rs1 - rs2$	
xor	0x4	0x00	0110011	$rd = rs1 \wedge rs2$	
or	0x6	0x00	0110011	$rd = rs1 \mid rs2$	
and	0x7	0x00	0110011	$rd = rs1 \& rs2$	
sll	0x1	0x00	0110011	$rd = rs1 \ll rs2$	
srl	0x5	0x00	0110011	$rd = rs1 \gg rs2$	
sra	0x5	0x20	0110011	$rd = rs1 \gg rs2$	msb-extends
slt	0x2	0x00	0110011	$rd = (rs1 < rs2)?1:0$	
sltu	0x3	0x00	0110011	$rd = (rs1 < rs2)?1:0$	zero-extends
mul	0x0	0x01	0110011	$rd = (rs1 * rs2)[31:0]$	
mulh	0x1	0x01	0110011	$rd = (rs1 * rs2)[63:32]$	
mulsu	0x2	0x01	0110011	$rd = (rs1 * rs2)[63:32]$	
mulu	0x3	0x01	0110011	$rd = (rs1 * rs2)[63:32]$	
div	0x4	0x01	0110011	$rd = rs1 / rs2$	
divu	0x5	0x01	0110011	$rd = rs1 / rs2$	
rem	0x6	0x01	0110011	$rd = rs1 \% rs2$	
remu	0x7	0x01	0110011	$rd = rs1 \% rs2$	

Fonte: O Autor

É importante destacar o comportamento de funções que trabalham com valores *unsigned*, valores que não apresentam sinais e que possuem características específicas, relacionadas a como eles são tratados quando estendidos. Tais comportamentos são descritos na coluna "Notas". A descrição *msb-extends* diz que o bit mais significativo é repetido quando o valor original é estendido, já a descrição *zero-extends* diz que o valor é completado por zeros quando é estendido. Essa afirmação também vale para tipos posteriores de instrução.

4.2.2 Instruções do Tipo I

As instruções do Tipo I (Immediate) caracterizam-se por operar combinando um registrador de origem a um valor constante (imediato) de 12 bits embutido na própria instrução, o qual sofre extensão de sinal prévia à sua utilização. Este formato é versátil e emprega dois modos de endereçamento distintos a depender da operação: utiliza o endereçamento por imediato para a execução de instruções lógico-aritméticas com constantes (a exemplo da soma com imediato) e o endereçamento por deslocamento para as instruções voltadas à leitura da memória de dados, utilizando o imediato como um referencial de distância (como nas operações de carregamento de palavra, meia palavra e byte). Além disso, este formato atende às necessidades de saltos indiretos realizados por valores de registradores.

Tabela 4 – Instruções do tipo I

Mnemônico	funct3	imm	Opcode	Descrição	Notas
addi	0x0		0010011	$rd = rs1 + imm$	
xori	0x4		0010011	$rd = rs1 \hat{=} imm$	
ori	0x6		0010011	$rd = rs1 imm$	
andi	0x7		0010011	$rd = rs1 \& imm$	
slli	0x1	$imm[5:11]=0x00$	0010011	$rd = rs1 \ll imm[0:4]$	
srli	0x5	$imm[5:11]=0x00$	0010011	$rd = rs1 \gg imm[0:4]$	
srai	0x5	$imm[5:11]=0x20$	0010011	$rd = rs1 \gg imm[0:4]$	msb-extends
slti	0x2		0010011	$rd = (rs1 < imm)?1:0$	
sltiu	0x3		0010011	$rd = (rs1 < imm)?1:0$	zero-extends
lb	0x0		0000011	$rd = M[rs1+imm][0:7]$	
lh	0x1		0000011	$rd = M[rs1+imm][0:15]$	
lw	0x2		0000011	$rd = M[rs1+imm][0:31]$	
lbu	0x4		0000011	$rd = M[rs1+imm][0:7]$	zero-extends
lhu	0x5		0000011	$rd = M[rs1+imm][0:15]$	zero-extends
jalr	0x0		1100111	$rd = PC+4; PC = rs1 + imm$	

Fonte: O Autor

4.2.3 Instruções do Tipo S

Para as operações de escrita, empregam-se as instruções do Tipo S (Store), que representam o formato exclusivo para a transferência de dados do processador para a memória externa. Estas operações fazem uso estrito do modo de endereçamento por deslocamento, no qual o endereço de destino na memória é computado através da soma do valor contido em um registrador base com um deslocamento imediato de 12 bits. Diferentemente das categorias anteriores, o Tipo S não requer um registrador de destino para gravação interna, uma vez que o dado operado é enviado ao sistema de armazenamento. Exemplos desta implementação englobam as operações de salvamento de palavra inteira, frações de palavra e unidades de byte.

Tabela 5 – Instruções do tipo S

Mnemônico	funct3	Opcode	Descrição
sb	0x0	0100011	$M[rs1+imm][0:7] = rs2[0:7]$
sh	0x1	0100011	$M[rs1+imm][0:15] = rs2[0:15]$
sw	0x2	0100011	$M[rs1+imm][0:31] = rs2[0:31]$

Fonte: O Autor

4.2.4 Instruções do Tipo B

No que tange à manipulação do fluxo de controle condicional, o sistema adota as instruções do Tipo B (Branch). Estas instruções operam calcadas no modo de endereçamento PC-relativo e são responsáveis por realizar avaliações lógicas entre dois registradores. Caso a condição avaliada seja considerada verdadeira, o processador efetua o desvio do fluxo de execução para um novo endereço, o qual é calculado mediante a soma do

valor atual do Contador de Programa (PC) a um valor imediato especificado na instrução. A implementação funcional deste projeto contempla saltos condicionais baseados em relações de igualdade, desigualdade e comparações de magnitude relativas.

Tabela 6 – Instruções do tipo B

Mnemônico	funct3	Opcode	Descrição	Notas
beq	0x0	1100011	if(rs1 == rs2) PC += imm	
bne	0x1	1100011	if(rs1 != rs2) PC += imm	
blt	0x4	1100011	if(rs1 < rs2) PC += imm	
bge	0x5	1100011	if(rs1 >= rs2) PC += imm	
bltu	0x6	1100011	if(rs1 < rs2) PC += imm	zero-extends
bgeu	0x7	1100011	if(rs1 >= rs2) PC += imm	zero-extends

Fonte: O Autor

4.2.5 Instruções do Tipo U

As instruções do tipo U são utilizadas para carregar constantes de 20 bits na parte superior de um registrador (`lui`) ou somá-las ao PC (`auipc`). O imediato de 20 bits é extraído dos bits [31:12] da instrução e deslocado 12 bits à esquerda.

Tabela 7 – Instruções do tipo U

Mnemônico	Opcode	Descrição
lui	0110111	rd = imm << 12
auipc	0010111	rd = PC + (imm << 12)

4.2.6 Instruções do Tipo J

A instrução `jal` (Jump and Link) é do tipo J, com um imediato de 20 bits (mais um bit 0 implícito) que permite saltos de até 51 MiB. O formato do imediato é reconstruído a partir dos campos `inst[31]`, `inst[19:12]`, `inst[20]`, `inst[30:21]`, acrescido de um bit 0 no LSB.

Tabela 8 – Instrução do tipo J

Mnemônico	Opcode	Descrição
jal	1101111	rd = PC+4; PC = PC + imm

4.3 Desenvolvimento da Organização

A organização do processador reflete a materialização física da arquitetura previamente definida, feita por meio da construção dos caminhos de dados de cada instrução, como da descrição do mesmo. Em cada componente funcional, há 4 campos:

1. **O nome do componente funcional**, localizado no centro de cada caixa.

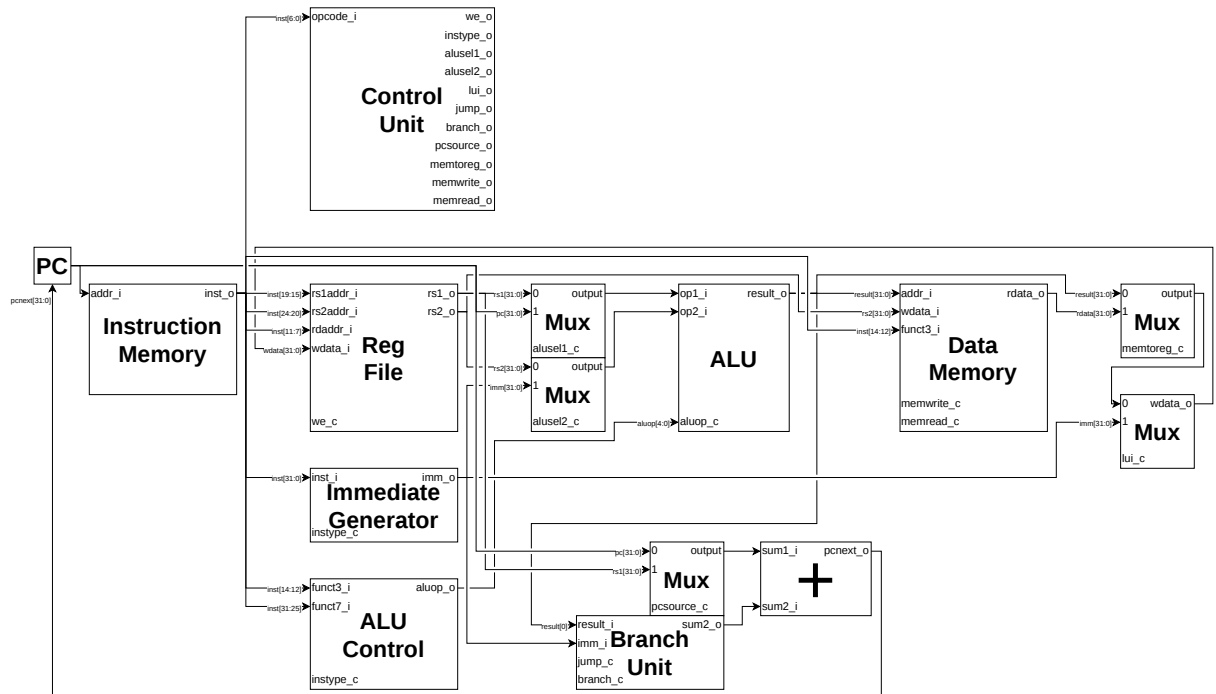
2. **As entradas gerais de cada componente funcional**, localizados no canto superior esquerdo de cada caixa e com nomes terminados em "`__i`".
3. **As entradas de controle de cada componente funcional**, localizadas no canto inferior esquerdo de cada caixa e com nomes terminados em "`__c`".
4. **As saídas de cada componente funcional**, localizadas na parte direita de cada caixa, com nomes terminados em "`__o`".

Vale ressaltar que para uma melhor compreensão de cada caminho de dados, foram omitidos os caminhos de controle com origem na unidade de controle (*Control Unit*), sendo representados por um texto em vermelho quando ativos, tanto na saída da unidade de controle quanto na entrada de controle dos componentes funcionais. Os caminhos são interligados por entradas e saídas com o mesmo nome antes do caractere "`__`".

Nos caminhos de dados de instruções específicas, além dos caminhos de controle, foram omitidos os caminhos não utilizados.

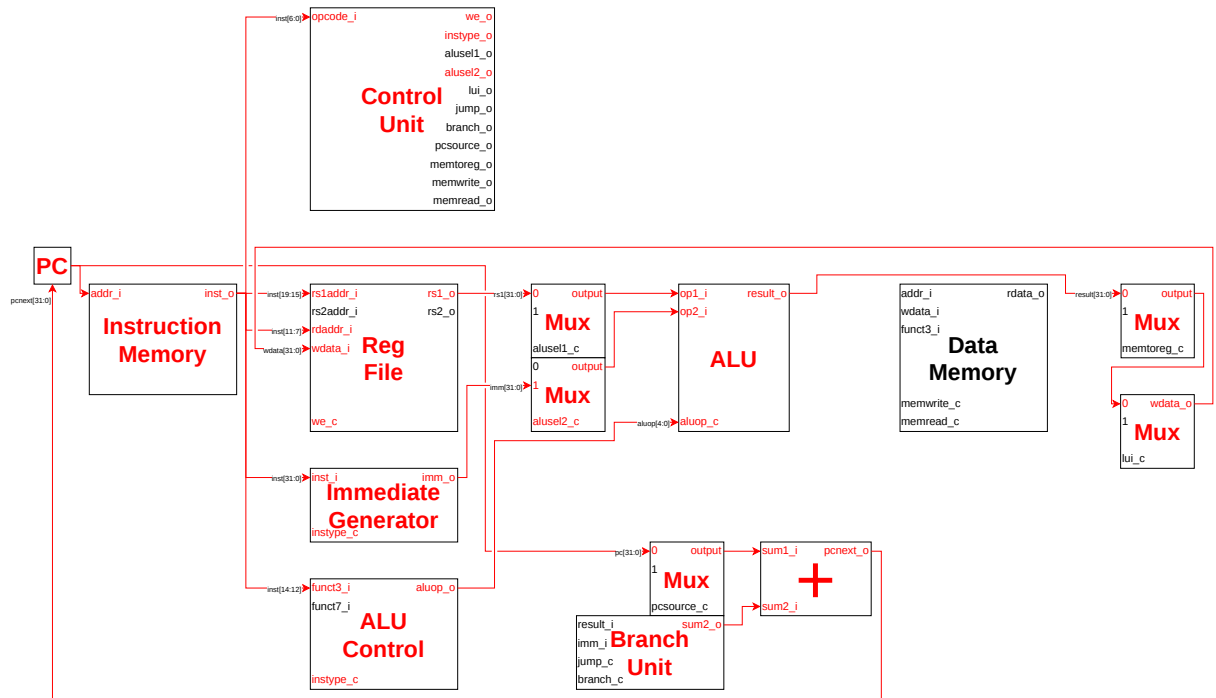
4.3.1 Caminho de Dados Completo

Figura 7 – Caminho de Dados Completo



Fonte: O Autor

Figura 9 – Caminho de Dados das Instruções Aritméticas e Lógicas com Imediato



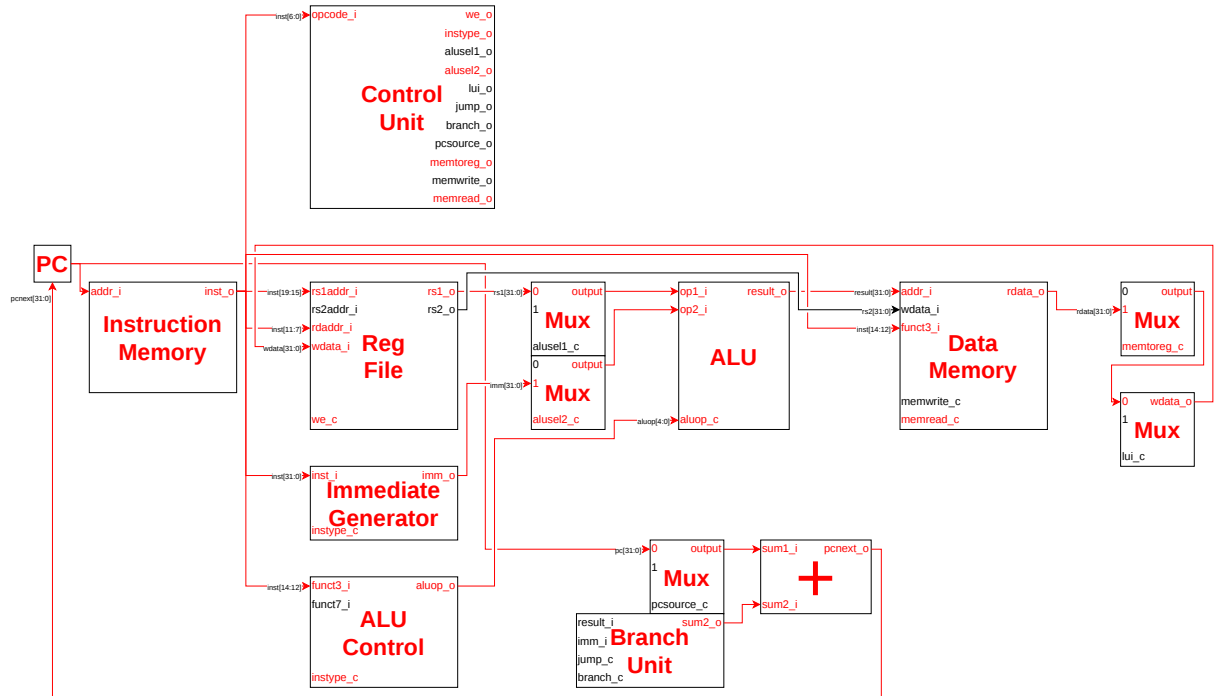
Fonte: O Autor

4.3.4 Caminho de Dados das Instruções de Carregamento (*lb*, *lh*, *lw*, *lbu*, *lhu*)

O fluxo de dados para a leitura em memória exige a utilização da ALU para o cálculo do endereço efetivo. O valor base é lido do registrador *rs1* e somado ao valor de deslocamento (imediato de 12 bits estendido). O resultado desta soma é enviado para a porta de endereço da Memória de Dados. Com o sinal de leitura ativado, a memória disponibiliza o dado requisitado. O multiplexador de write-back é então configurado para selecionar a saída da Memória de Dados, redirecionando o valor lido para ser armazenado no registrador de destino (*rd*).

É importante ressaltar que para as instruções sem bit de sinal *lbu* e *lhu*, a extensão do valor é realizada dentro da própria memória de dados. O reconhecimento de tais instruções é realizado de mesma forma do reconhecimento das demais instruções de carregamento, utilizando-se do campo *funct3* da instrução.

Figura 10 – Caminho de Dados das Instruções de Carregamento

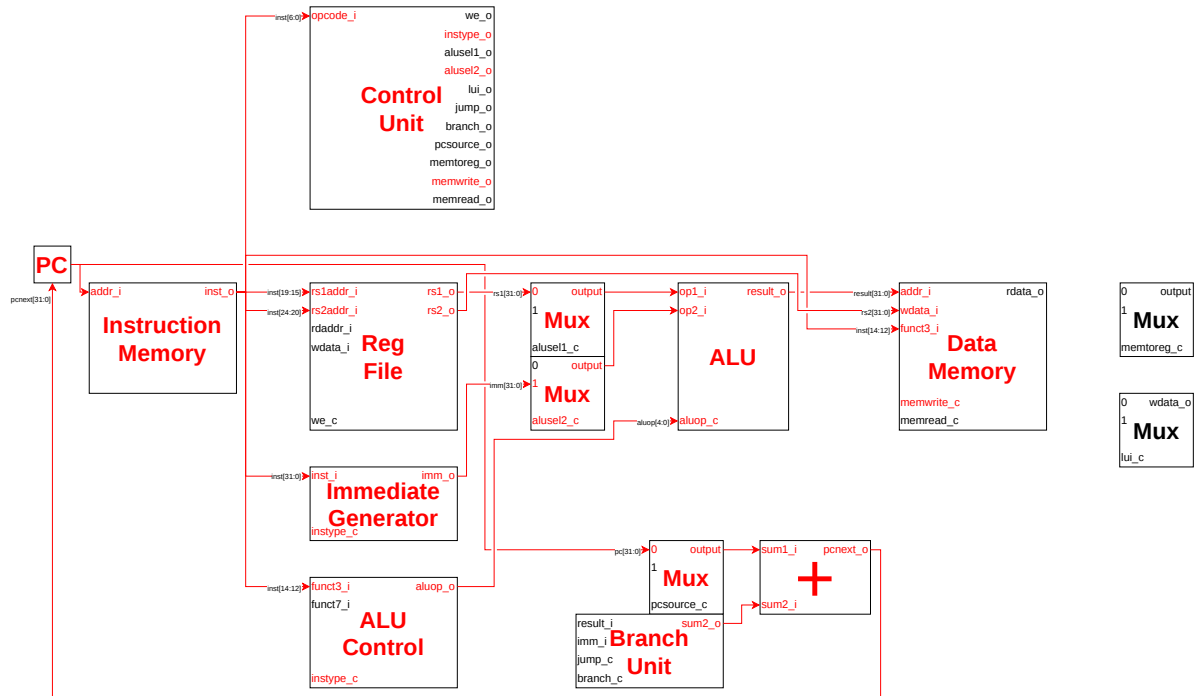


Fonte: O Autor

4.3.5 Caminho de Dados das Instruções de Armazenamento (*sb*, *sh*, *sw*)

Para as operações de escrita na memória, o datapath processa dois registradores simultaneamente: *rs1* fornece o endereço base e *rs2* fornece o dado a ser armazenado. A ALU computa o endereço final de destino somando o valor de *rs1* ao imediato estendido. O resultado da ALU é ligado à entrada de endereçamento da Memória de Dados, enquanto o valor lido de *rs2* é direcionado para a porta de dados de entrada da memória. O sinal de escrita em memória é habilitado e, como não há retorno de dados para o processador, o sinal de escrita no Banco de Registradores é desativado.

Figura 11 – Caminho de Dados das Instruções de Armazenamento

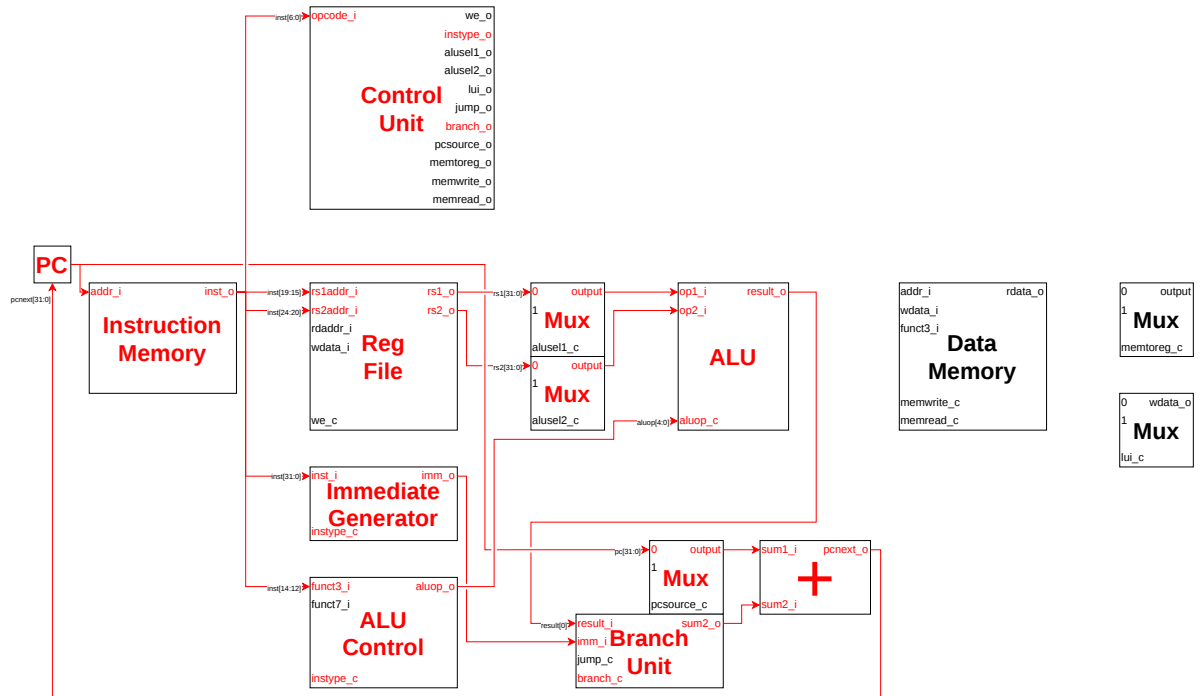


Fonte: O Autor

4.3.6 Caminho de Dados das Instruções de Desvio Condicional (*beq*, *bne*, *blt*, *bge*, *bltu*, *bgeu*)

Nas instruções de salto condicional, o caminho de dados bifurca-se em duas operações simultâneas. Na primeira via, os valores dos registradores *rs1* e *rs2* são enviados à ALU (ou a um comparador dedicado) para avaliar a condição lógica estipulada. Na segunda via, um somador independente calcula o endereço de destino do salto, adicionando o valor atual do Contador de Programa (PC) ao imediato de 12 bits estendido (e deslocado à esquerda). Se a condição avaliada pela ALU for verdadeira, a Unidade de Controle comuta o multiplexador do PC para que o próximo endereço de instrução seja o alvo calculado; caso contrário, o PC é atualizado normalmente para $PC + 4$. Não ocorre gravação no Banco de Registradores.

Figura 12 – Caminho de Dados das Instruções de Desvio Condicional

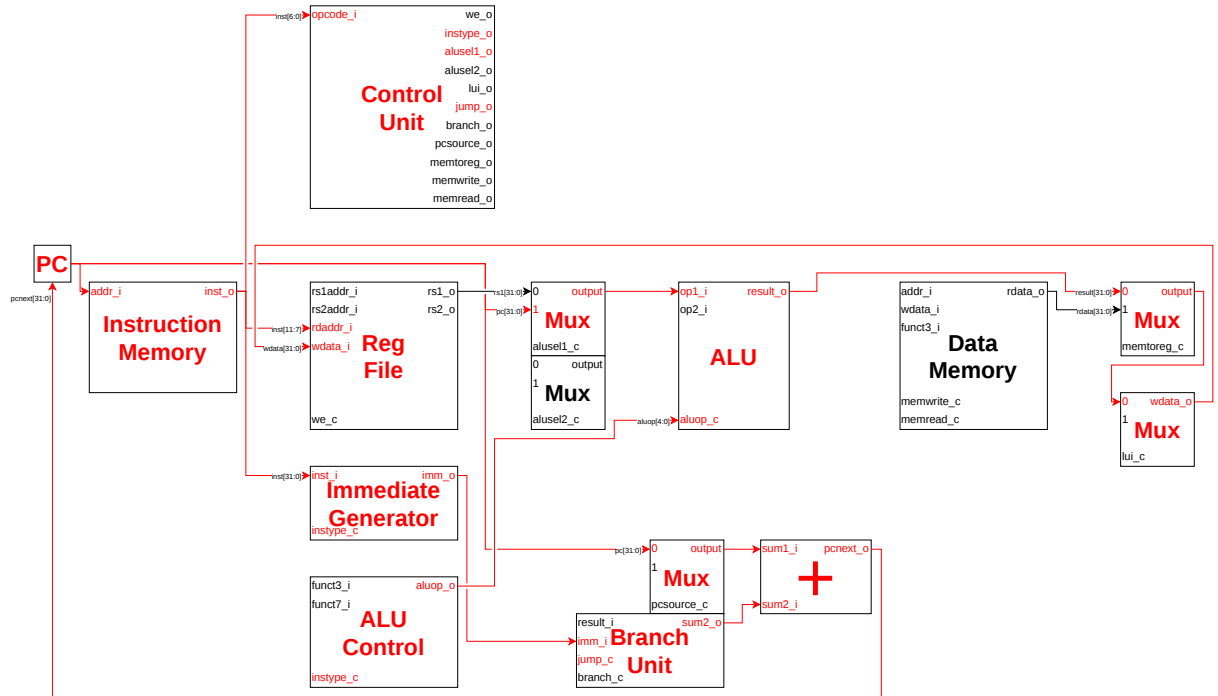


Fonte: O Autor

4.3.7 Caminho de Dados da Instrução de Salto e Ligação (*jal*)

O processamento da instrução *jal* requer o cálculo do endereço de salto e do endereço de retorno. O endereço de destino é obtido somando-se o valor atual do PC ao imediato de 20 bits estendido presente na instrução. Esse valor é injetado diretamente no PC. Concomitantemente, o endereço sequencial da próxima instrução ($PC + 4$) é calculado na ALU e enviado para a porta de escrita do Banco de Registradores, sendo salvo no registrador destino (*rd*, convencionalmente o registrador de retorno), permitindo a posterior retomada do fluxo original.

Figura 13 – Caminho de Dados da Instrução de Salto e Ligação

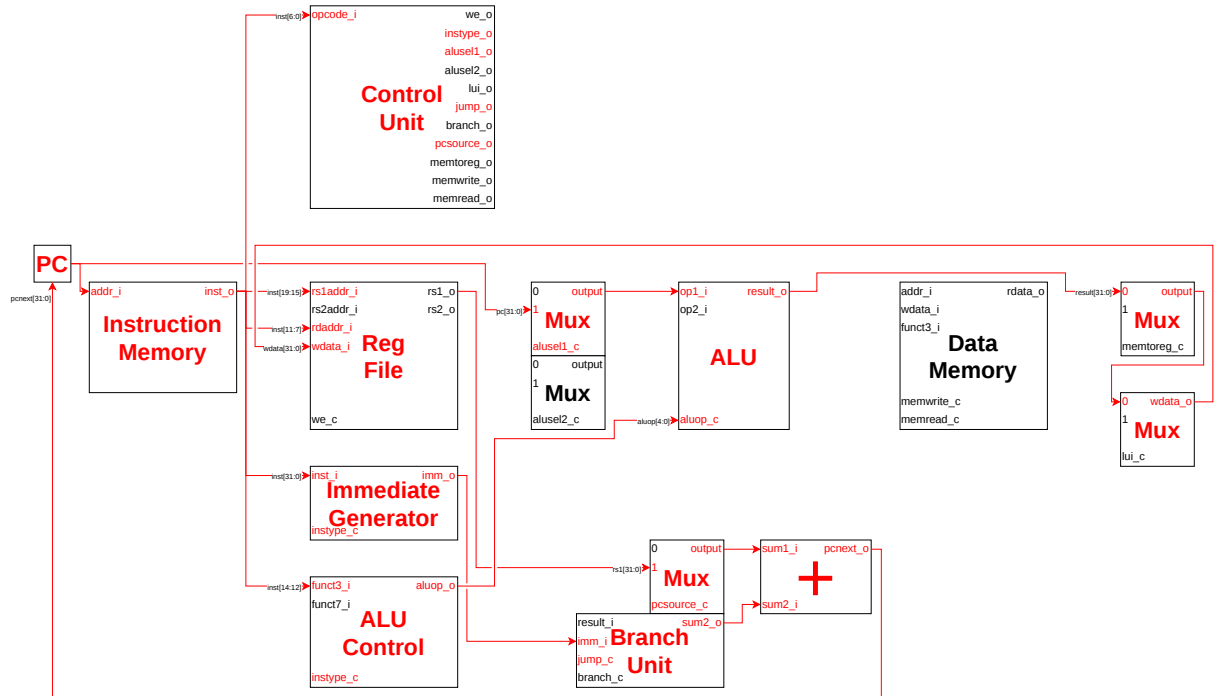


Fonte: O Autor

4.3.8 Caminho de Dados da Instrução de Salto Indireto (*jalr*)

A execução do *jalr* assemelha-se estruturalmente ao *jal*, porém com uma diferença crucial no cálculo do endereço de destino. Em vez de utilizar o PC como base, o endereço do salto é calculado pela ALU somando o valor lido do registrador *rs1* com o imediato de 12 bits com sinal estendido. O endereço de retorno ($PC + 4$) é preservado e gravado no registrador destino (*rd*), e o PC é atualizado com o novo endereço calculado, permitindo saltos dinâmicos baseados em dados armazenados.

Figura 14 – Caminho de Dados da Instrução de Salto Indireto

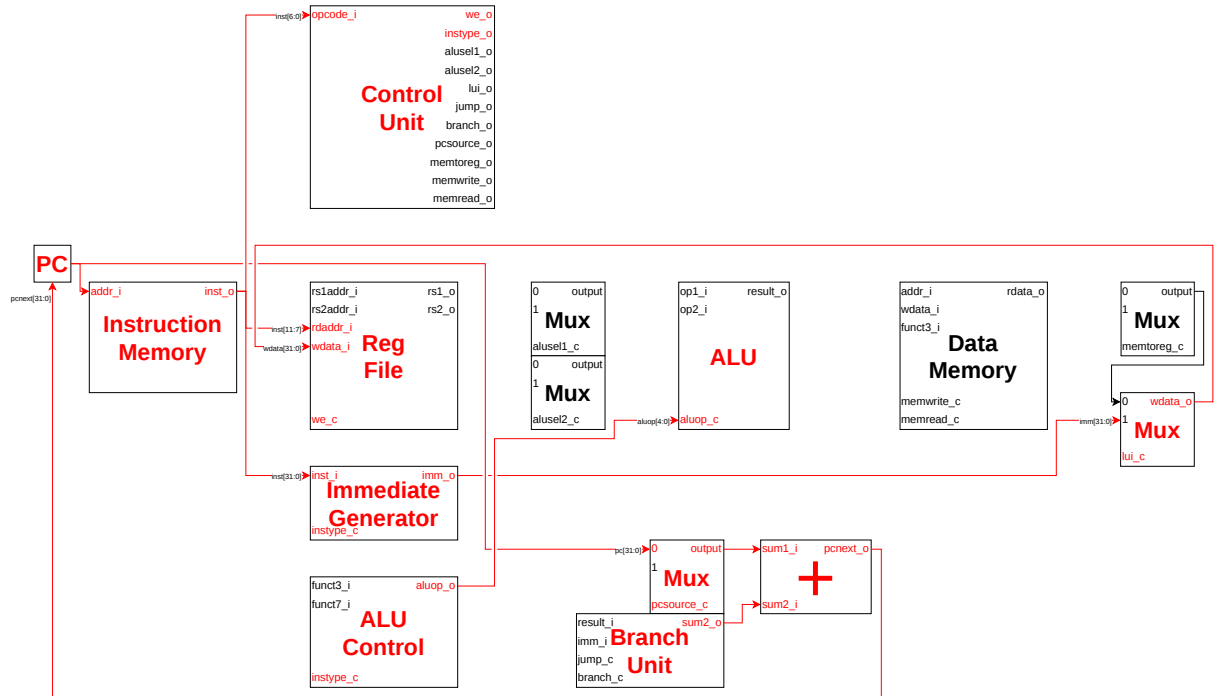


Fonte: O Autor

4.3.9 Caminho de Dados de Carregamento de Imediato Superior (*lui*)

Para a instrução *lui*, o datapath opera de maneira simplificada. O Gerador de Imediatos extrai os 20 bits superiores da instrução, preenche os 12 bits inferiores com zeros e envia este valor de 32 bits pelo caminho de dados e o resultado é imediatamente conduzido ao Banco de Registradores para ser gravado em *rd*. Não há acesso à memória de dados nem leitura de registradores de origem.

Figura 15 – Caminho de Dados de Carregamento de Imediato Superior

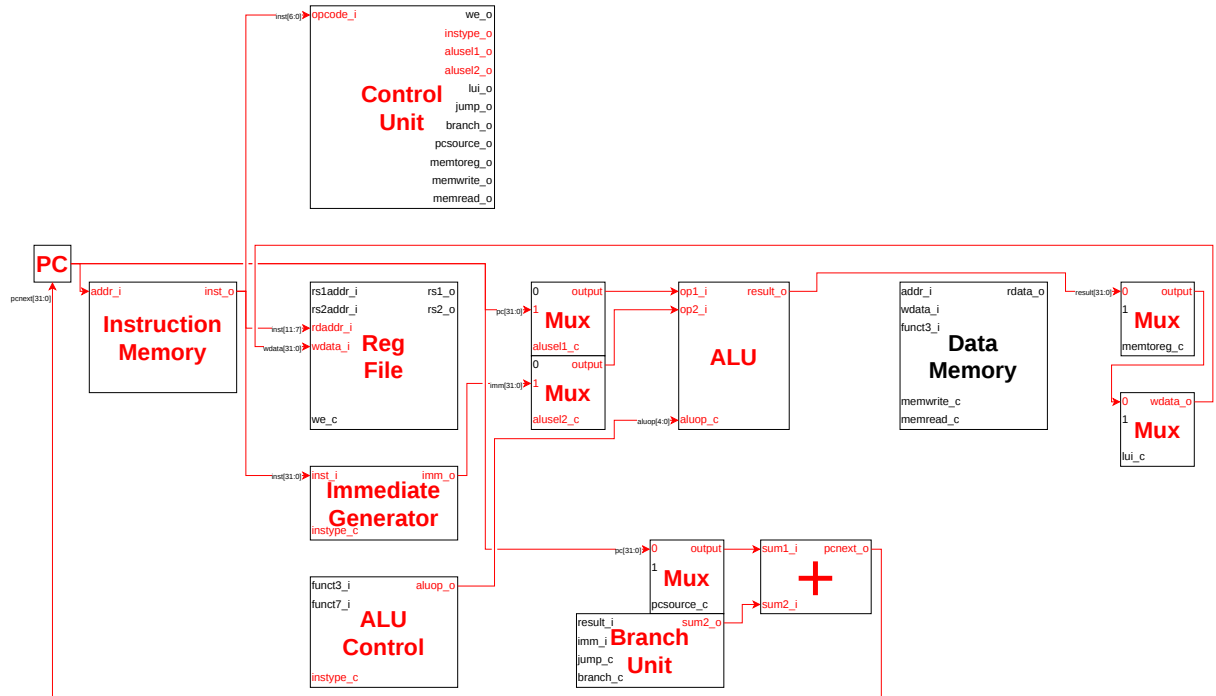


Fonte: O Autor

4.3.10 Caminho de Dados da Adição de Imediato Superior ao PC (*auipc*)

Por fim, o fluxo da instrução *auipc* combina o valor atual do PC com uma constante. O imediato de 20 bits é extraído e posicionado nas casas superiores de uma palavra de 32 bits (com os bits inferiores zerados). A ALU recebe em suas entradas o valor do PC e este imediato formatado, realizando a soma de ambos. O endereço resultante desta adição é então roteado diretamente para a porta de escrita do Banco de Registradores, sendo salvo em *rd* para utilização futura pelo programa.

Figura 16 – Caminho de Dados da Adição de Imediato Superior ao PC



Fonte: O Autor

4.4 Definições Globais e Package data_structures

Antes da implementação dos módulos funcionais, foi criado um package em System-Verilog `data_structures.sv` para centralizar todas as definições de tipos, enumeradores e constantes utilizadas no projeto. Esse package é compilado antes dos demais módulos, garantindo consistência e facilitando a manutenção.

```

1 package data_structures;
2   typedef logic [31:0] register_t;
3   typedef logic [31:0] instruction_t;
4   typedef logic [31:0] address_t;
5   typedef logic [31:0] data_t;
6
7   typedef logic [4:0] reg_addr_t;
8   typedef logic [6:0] funct7_t;
9   typedef logic [2:0] funct3_t;
10
11  typedef enum logic [2:0] {
12    OP      = 3'b000,
13    OPI     = 3'b001,
14    AUIPC   = 3'b010,
15    BRANCH  = 3'b011,
16    JUMP    = 3'b100,
17    LOAD    = 3'b101,
18    STORE   = 3'b110
19  } inst_type_t;
20
21  typedef enum logic [6:0] {

```

```

22     opcode_OP      = 7'b0110011,
23     opcode_OP_IMM  = 7'b0010011,
24     opcode_LOAD    = 7'b0000011,
25     opcode_STORE   = 7'b0100011,
26     opcode_BRANCH  = 7'b1100011,
27     opcode_JAL     = 7'b1101111,
28     opcode_JALR    = 7'b1100111,
29     opcode_LUI     = 7'b0110111,
30     opcode_AUIPC   = 7'b0010111
31 } opcode_t;
32
33 typedef enum logic [4:0] {
34     ADD = 5'b00000, SUB = 5'b00001, XOR = 5'b00010, OR = 5'b00011,
35     AND = 5'b00100, SLL = 5'b00101, SRL = 5'b00110, SRA = 5'b00111,
36     SLT = 5'b01000, SLTU = 5'b01001, EQUAL = 5'b01010, NEQUAL = 5'b01011,
37     GOET = 5'b01100, GETU = 5'b01101, PCP4 = 5'b01110,
38     MUL = 5'b01111, MULH = 5'b10000, MULHSU = 5'b10001, MULHU = 5'b10010,
39     DIV = 5'b10011, DIVU = 5'b10100, REM = 5'b10101, REMU = 5'b10110
40 } aluop_t;
41 endpackage

```

Listing 4.1 – Package data_structures.sv

As enumerações `inst_type_t` e `aluop_t` são particularmente importantes, pois mapeiam os tipos de instrução e as operações da ULA a nomes simbólicos, tornando o código mais legível e próximo da especificação RISC-V.

4.5 Desenvolvimento da Unidade Lógica e Aritmética (ALU)

A Unidade Lógica e Aritmética é o componente central responsável por executar todas as operações matemáticas, lógicas e de comparação exigidas pelo conjunto de instruções do processador. Neste projeto, a ALU foi projetada para suportar não apenas as operações da base inteira (RV32I), mas também a extensão de multiplicação e divisão (RV32M), totalizando 24 operações distintas. O módulo foi implementado em Verilog (arquivo `alu.sv`) seguindo o modelo de lógica combinacional pura, com blocos `always_comb` para o controle e para a execução das operações.

4.5.1 Interface da ALU

A interface da ALU é definida pelo seguinte conjunto de portas (conforme trecho do código):

```

1 module alu (
2     input data_t rs1_i, rs2_i, imm_i, pc_i,
3     input funct3_t funct3_i,
4     input funct7_t funct7_i,
5     input inst_type_t inst_type_c,
6     input logic alusel1_c, alusel2_c,
7     output data_t aluout_o

```

8);

Listing 4.2 – Interface da ALU

As entradas de dados (`rs1_i`, `rs2_i`, `imm_i`, `pc_i`) são os valores que podem ser selecionados como operandos da ALU. Os sinais de controle `alusel1_c` e `alusel2_c` permitem escolher, respectivamente, se o primeiro operando será `rs1_i` ou `pc_i` e se o segundo operando será `rs2_i` ou `imm_i`. O campo `inst_type_c` indica o tipo de instrução (OP, OPI, LOAD, STORE, BRANCH, JUMP, AUIPC), enquanto `funct3_i` e `funct7_i` são campos da instrução que refinam a operação. A saída `aluout_o` fornece o resultado calculado.

4.5.2 Decodificação da Operação Mapeamento para `aluop_t`

Para abstrair a variedade de combinações entre `inst_type`, `funct3` e `funct7`, foi criado um tipo enumerado `aluop_t` (definido no package `data_structures.sv`) com todos os códigos de operação suportados:

```
1 typedef enum logic [4:0] {
2     ADD, SUB, XOR, OR, AND, SLL, SRL, SRA, SLT, SLTU,
3     EQUAL, NEQUAL, GOET, GETU, PCP4,
4     MUL, MULH, MULHSU, MULHU, DIV, DIVU, REM, REMU
5 } aluop_t;
```

Listing 4.3 – Decodificação da operação

O bloco ALUcontrol utiliza um casez para mapear cada combinação relevante ao respectivo `aluop`. Exemplos:

- Para instruções do tipo R aritméticas (OP):

```
1 {OP, 3'd0, 7'd0} : aluop = ADD;
2 {OP, 3'd0, 7'd32} : aluop = SUB;
```

- Para instruções de desvio condicional (BRANCH):

```
1 {BRANCH, 3'd0, 7'b???????} : aluop = EQUAL;
2 {BRANCH, 3'd4, 7'b???????} : aluop = SLT; // (para blt)
```

- Para multiplicação e divisão (extensão M):

```
1 {OP, 3'd0, 7'd1} : aluop = MUL;
2 {OP, 3'd4, 7'd1} : aluop = DIV;
```

Observe o uso de dont-cares (?) em campos que não influenciam a operação, como nos casos OPI onde o `funct7` é irrelevante, exceto para deslocamentos com parte imediata.

4.5.3 Geração dos Operandos

Os dois operandos efetivos (`op_1` e `op_2`) são gerados por um bloco combinacional que utiliza os sinais de seleção:

```
1 op_1 = (alusel1_c) ? pc_i : rs1_i;
2 op_2 = (alusel2_c) ? imm_i : rs2_i;
```

Essa lógica permite que a ALU atenda a todos os modos de endereçamento necessários:

- **Tipo R:** `alusel1_c = 0`, `alusel2_c = 0` `op_1 = rs1`, `op_2 = rs2`.
- **Aritmética com imediato (OPI):** `alusel1_c = 0`, `alusel2_c = 1` `op_1 = rs1`, `op_2 = imm`.
- **Cálculo de endereço para LOAD/STORE:** `alusel1_c = 0`, `alusel2_c = 1` `op_1 = rs1`, `op_2 = imm` (ADD).
- **JAL (salto e ligação):** `alusel1_c = 1`, `alusel2_c = 0` `op_1 = pc`, `op_2 = rs2` (não usado), mas a operação PCP4 calcula `pc + 4` independentemente.
- **AUIPC:** `alusel1_c = 1`, `alusel2_c = 1` `op_1 = pc`, `op_2 = imm`, operação ADD.

4.5.4 Implementação das Operações

A execução propriamente dita ocorre em um terceiro bloco `always_comb` que contém um case (`aluop`). As principais operações são implementadas da seguinte forma:

- **Aritméticas e lógicas básicas:** ADD, SUB, XOR, OR, AND usam operadores do SystemVerilog diretamente.
- **Deslocamentos:** SLL, SRL e SRA consideram apenas os 5 bits menos significativos do segundo operando (`op_2[4:0]`), conforme especificação RISC-V. O deslocamento aritmético para a direita utiliza

```
1    $signed(op_1) >>> op_2[4:0].
```

- **Comparações com sinal e sem sinal:** SLT e SLTU produzem 1 no bit 0 do resultado se a condição for verdadeira.
- **Comparações para desvios:** EQUAL, NEQUAL, GOET (greater or equal), GETU (unsigned greater or equal) também produzem um bit de condição no LSB.
- **Operações de multiplicação e divisão:**
 - MUL produto de 64 bits truncado para os 32 bits inferiores.

- MULH, MULHSU, MULHU respectivamente, multiplicação com sinal (E sinal, sinal E sem sinal, sem sinal E sem sinal, retornando os 32 bits superiores.
 - DIV divisão com sinal, com tratamento especial para a divisão por zero (resultado = -1) e para o caso $0x80000000 / -1$ (resultado = $0x80000000$).
 - DIVU divisão sem sinal, retornando $0xFFFFFFFF$ para divisor zero.
 - REM e REMU resto da divisão.
- **Opção auxiliar PCP4:** utilizada pelas instruções jal e jalr para calcular o endereço de retorno ($pc + 4$).

Todos os cálculos que exigem largura de 64 bits utilizam o registrador temporário tmp_reg de 64 bits para armazenar o produto antes da seleção da parte desejada.

4.5.5 Desenvolvimento dos Testbenches

Para validar o funcionamento da ALU, foi elaborado um testbench específico (tb.sv) que instancia o módulo alu e aplica estímulos correspondentes a todas as classes de instruções suportadas. O testbench é modular e permite a ativação seletiva de cada grupo de testes, facilitando a depuração e a verificação incremental.

4.5.5.1 Estrutura do Testbench

O testbench declara os mesmos sinais da interface da ALU e os conecta ao módulo sob teste (dut). Um bloco inicial é responsável por:

1. Inicializar todos os sinais com valores padrão.
2. Executar, em sequência, os conjuntos de testes correspondentes a:
 - Instruções do tipo R aritméticas (ADD, SUB, MUL, DIV, etc.)
 - Instruções do tipo R de deslocamento (SLL, SRL, SRA)
 - Instruções do tipo R lógicas (XOR, OR, AND, SLT, SLTU)
 - Instruções do tipo I (ADDI, XORI, ORI, ANDI, SLLI, SRLI, SRAI, SLTI, SLTIU)
 - Instruções de LOAD e STORE (cálculo de endereço)
 - Instruções de desvio condicional (BEQ, BNE, BLT, BGE, BLTU, BGEU)
 - Instruções de salto e ligação (JAL, JALR) e AUIPC

Cada teste configura os sinais de entrada adequados e aguarda um intervalo de 10 unidades de tempo (#10) para que a ALU compute o resultado. Ao final, uma chamada \$stop pausa a simulação, permitindo a análise das formas de onda.

4.5.5.2 Organização dos Estímulos

Para manter a clareza, os testes foram agrupados e podem ser ativados/desativados através de comentários no código. Por exemplo, o trecho para as instruções do tipo R aritméticas é:

```

1 inst_type_c = OP;
2 alusel1_c = 0;
3 alusel2_c = 0;
4
5 // ADD
6 rs1_i = 32'd15; rs2_i = 32'd25; funct3_i = 3'd0; funct7_i = 7'd0; #10;
7 // SUB
8 rs1_i = 32'd50; rs2_i = 32'd20; funct3_i = 3'd0; funct7_i = 7'd32; #10;
9 // MUL
10 rs1_i = 32'd1000000; rs2_i = 32'd1000000; funct3_i = 3'd0; funct7_i = 7'd1; #10;
11 // ... etc.
```

Listing 4.4 – Exemplo do trecho do código de instruções do tipo R aritméticas no TestBench

Para as instruções do tipo I, além de configurar `inst_type_c = OPI` e `alusel2_c = 1`, o campo `imm_i` é utilizado:

```

1 // ADDI
2 rs1_i = 32'd100; imm_i = 32'd75; funct3_i = 3'd0; #10;
3 // SRAI
4 rs1_i = -32'd1024; imm_i = 32'd1; funct3_i = 3'd5; funct7_i = 7'd32; #10;
```

Listing 4.5 – Exemplo do trecho do código de instruções do tipo I aritméticas no TestBench

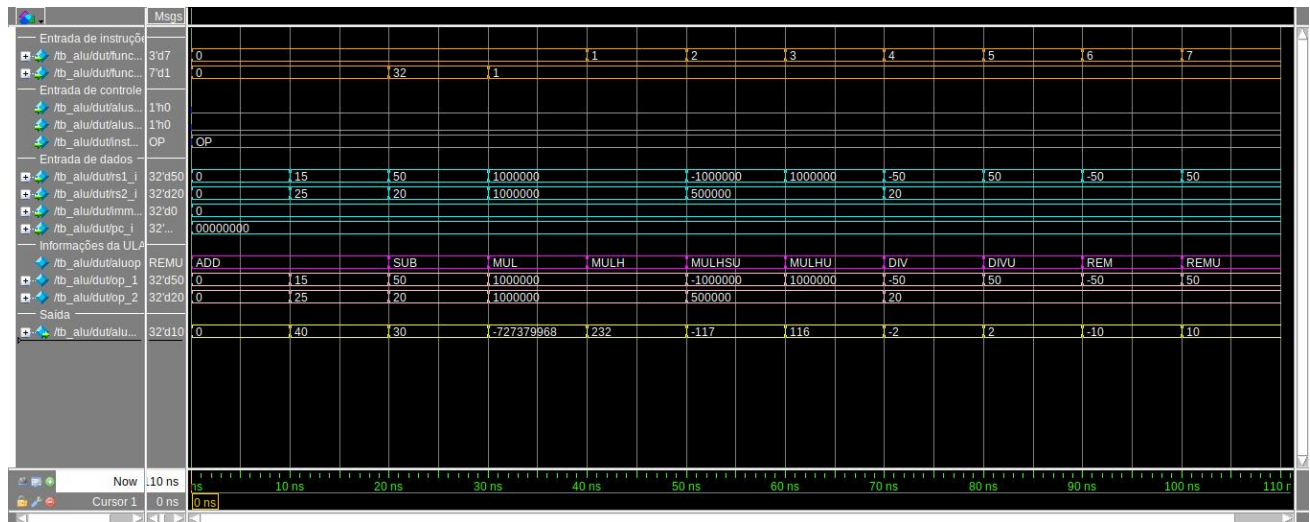
Nos testes de desvio condicional, a saída da ALU fornece um bit de condição no LSB (`aluout_o[0]`), que é posteriormente usado pela unidade de controle para decidir o salto. O testbench apenas verifica a geração correta desse bit.

4.5.6 Resultados da Simulação da ALU

4.5.6.1 Instruções do tipo R

4.5.6.1.1 Aritméticas

Figura 17 – Waveform das instruções do tipo R aritméticas (ADD, SUB, MUL, MULH, MULHSU, MULHU, DIV, DIVU, REM, REMU)



Fonte: O Autor

Observase na figura que o testbench configura inicialmente `inst_type_c = OP`, `alusel1_c = 0` e `alusel2_c = 0`, selecionando a operação entre dois registradores. Os sinais `funct3_i` e `funct7_i` variam a cada intervalo de 10 ns para executar diferentes operações.

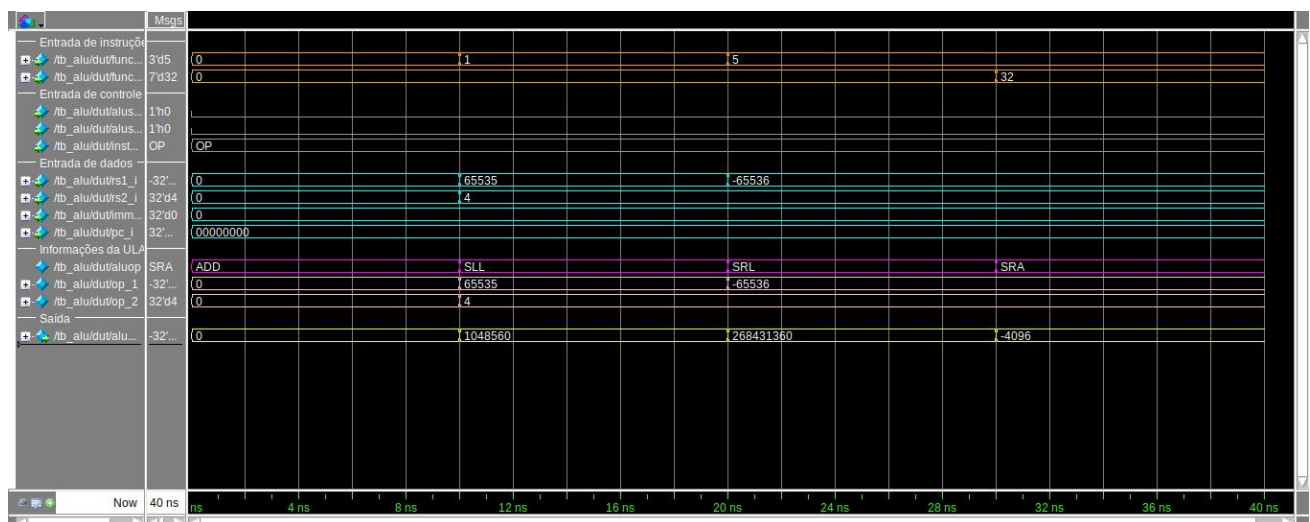
- **ADD** (`funct3=0`, `funct7=0`): `rs1_i = 15`, `rs2_i = 25`. A ALU decodifica `aluop = ADD` e a saída `aluout_o` tornase **40** (decimal), conforme esperado.
- **SUB** (`funct3=0`, `funct7=32`): `rs1_i = 50`, `rs2_i = 20`. `aluop = SUB` e `aluout_o = 30`.
- **MUL** (`funct3=0`, `funct7=1`): `rs1_i = 1.000.000`, `rs2_i = 1.000.000`. O resultado da multiplicação de 32 bits é a parte baixa do produto de 64 bits. O waveform mostra `aluout_o = 0x4A5C1000` (valor decimal $1.000.000^2 \bmod 2^{32}$). A operação MUL está correta.
- **MULH** (`funct3=1`, `funct7=1`): Mesmos operandos. `aluout_o` assume o valor `0xE8D4A510` (parte alta do produto com sinal). O waveform indica que a ALU seleciona `tmp_reg[63:32]` corretamente.

- **MULHSU** (funct3=2, funct7=1): rs1_i = -1.000.000 (complemento de dois), rs2_i = 500.000. A saída corresponde aos 32 bits superiores do produto de um número com sinal por um sem sinal. O valor exibido está de acordo com a simulação.
- **MULHU** (funct3=3, funct7=1): rs1_i = 1.000.000, rs2_i = 500.000. Produto sem sinal, parte alta. Verificado.
- **DIV** (funct3=4, funct7=1): rs1_i = -50, rs2_i = 20. aluout_o = -2 (quociente com sinal). Na waveform, o valor aparece como -32'd2 (ou 0xFFFFFFF2).
- **DIVU** (funct3=5, funct7=1): rs1_i = 50, rs2_i = 20. aluout_o = 2 (divisão sem sinal).
- **REM** (funct3=6, funct7=1): rs1_i = -50, rs2_i = 20. Resto = -10. O waveform confirma aluout_o = -10.
- **REMU** (funct3=7, funct7=1): rs1_i = 50, rs2_i = 20. Resto = 10.

A ALU executa corretamente todas as operações aritméticas da extensão M (multiplicação e divisão) além das básicas ADD e SUB.

4.5.6.1.2 Shift

Figura 18 – Waveform das instruções de deslocamento (SLL, SRL, SRA)



Fonte: O Autor

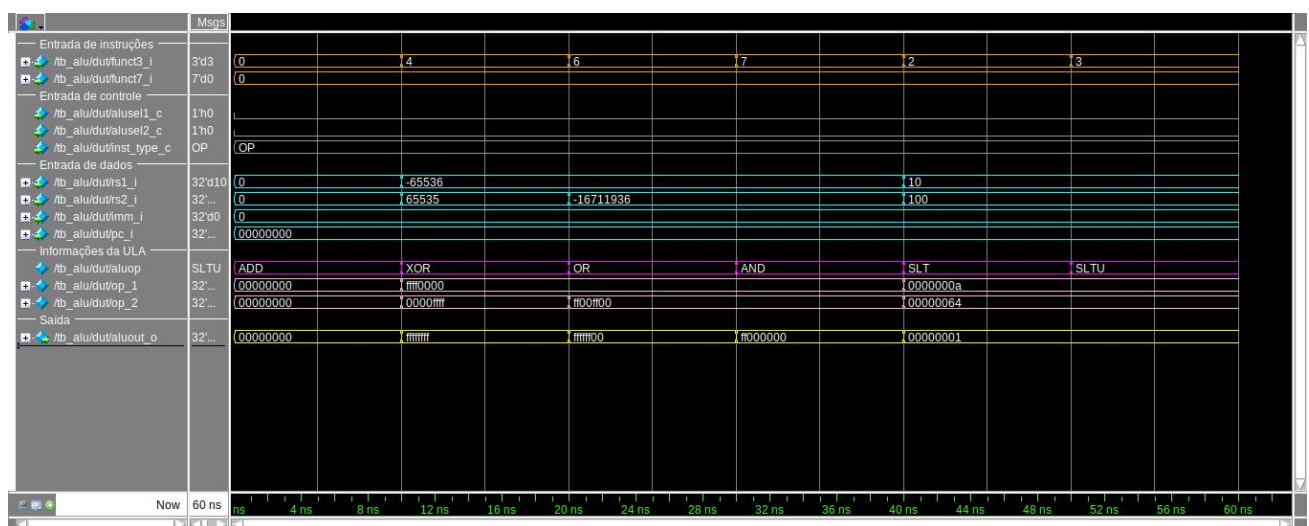
- **SLL** (funct3=1, funct7=0): rs1_i = 32'h0000FFFF, rs2_i = 4. Operação shift left lógico: $0x0000FFFF \ll 4 = 0x000FFFF0$. O waveform mostra aluout_o = 0x000FFFF0.

- **SRL** (funct3=5, funct7=0): $rs1_i = 32'hFFFF0000$, $rs2_i = 4$. Shift right lógico: $0xFFFF0000 \gg 4 = 0x0FFF0000$. A saída confirma esse valor.
- **SRA** (funct3=5, funct7=32): O mesmo $rs1_i = 0xFFFF0000$ (valor negativo em complemento de dois) deslocado aritmeticamente à direita por 4 posições. O bit de sinal (MSB = 1) é replicado, resultando em $0xFFFFF000$. O waveform exhibe exatamente $0xFFFFF000$.

A ALU trata corretamente os deslocamentos lógicos e aritméticos, considerando apenas os 5 bits inferiores do segundo operando.

4.5.6.1.3 Lógicas

Figura 19 – Instruções do Tipo R Lógicas e Comparações



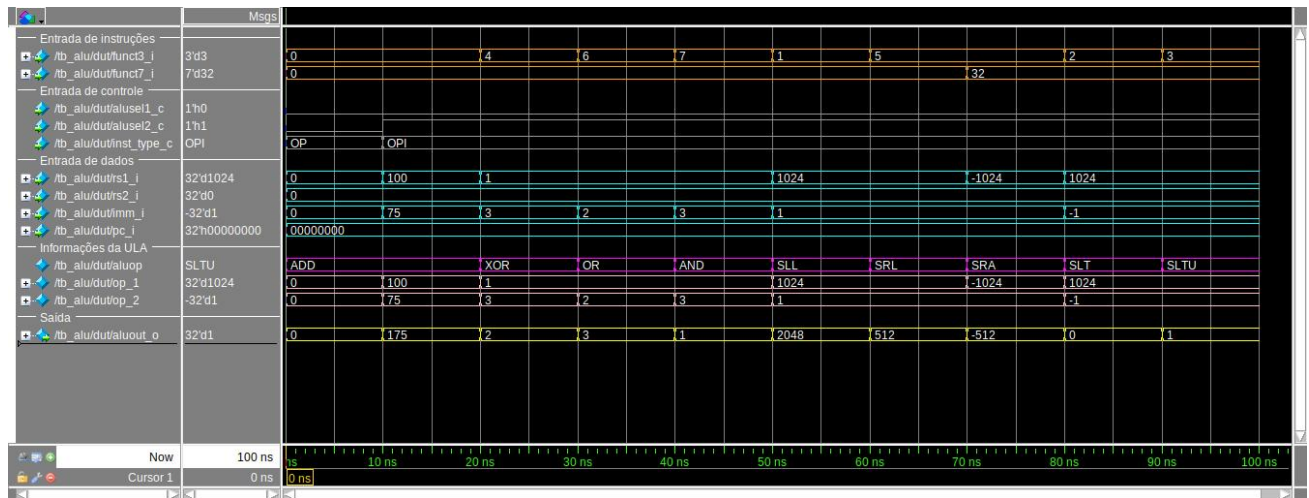
Fonte: O Autor

- **XOR** (funct3=4, funct7=0): $rs1_i = 0xFFFF0000$, $rs2_i = 0x0000FFFF$. Resultado = $0xFFFFFFFF$. Saída correta.
- **OR** (funct3=6, funct7=0): $rs1_i = 0xFFFF0000$, $rs2_i = 0xFF00FF00$. OR bit a bit = $0xFFFFFFFF00$. O waveform mostra esse valor.
- **AND** (funct3=7, funct7=0): Mesmos operandos AND = $0xFF000000$. Confirmado.
- **SLT** (funct3=2, funct7=0): $rs1_i = 10$, $rs2_i = 100$ (comparação com sinal). Como $10 < 100$, $aluout_o[0] = 1$. O waveform exhibe $aluout_o = 1$ (apenas o LSB é significativo).

- **SLTU** (funct3=3, funct7=0): Mesmos valores, mas comparação sem sinal. $10 < 100$ também 1. Confirma-se.

4.5.6.2 Instruções do Tipo I (Aritmética com Imediato)

Figura 20 – Waveform das instruções do tipo I (ADDI, XORI, ORI, ANDI, SLLI, SRLI, SRAI, SLTI, SLTIU).



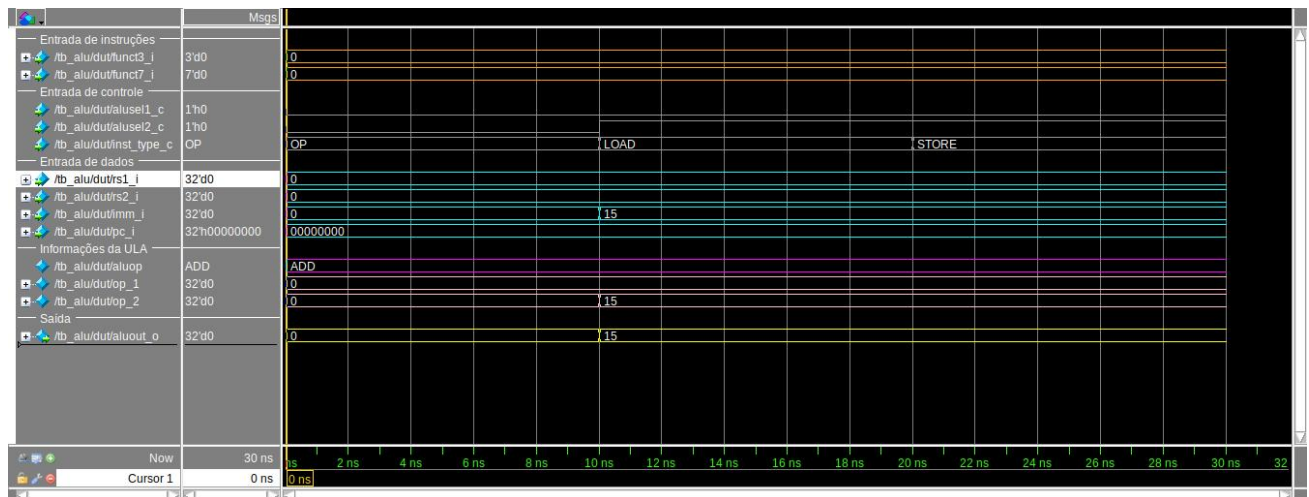
Fonte: O Autor

- **ADDI:** `rs1_i = 100, imm_i = 75` `aluout_o = 175`.
- **XORI:** `rs1_i = 1, imm_i = 3` `1 XOR 3 = 2`. Saída = 2.
- **ORI:** `rs1_i = 1, imm_i = 2` `1 OR 2 = 3`.
- **ANDI:** `rs1_i = 1, imm_i = 3` `1 AND 3 = 1`.
- **SLLI:** `rs1_i = 1024, imm_i = 1` (shift left por 1) `2048`. O campo `funct7` é 0. Saída = 2048.
- **SRLI:** `rs1_i = 1024, imm_i = 1` (shift right lógico) `512`. Saída = 512.
- **SRAI:** `rs1_i = -1024` (em complemento de dois), `imm_i = 1` deslocamento aritmético à direita `-512`. O waveform mostra `aluout_o = -512` (ou `0xFFFFFE00`).
- **SLTI:** `rs1_i = 1024, imm_i = -1`. Comparação com sinal: $1024 < -1$? Falso `aluout_o[0] = 0`.
- **SLTIU:** `rs1_i = 1024, imm_i = -1`. Agora `-1` é interpretado como um número sem sinal (`0xFFFFFFFF`). $1024 < 0xFFFFFFFF$ verdadeiro `aluout_o[0] = 1`.

A ALU decodifica corretamente as instruções de imediato, inclusive com a extensão de sinal do campo `imm_i` e o tratamento especial para deslocamentos (que usam `funct7` para distinguir SRLI de SRAI).

4.5.6.3 Instruções de Load e Store

Figura 21 – Waveform do cálculo de endereço para LOAD/STORE.



Fonte: O Autor

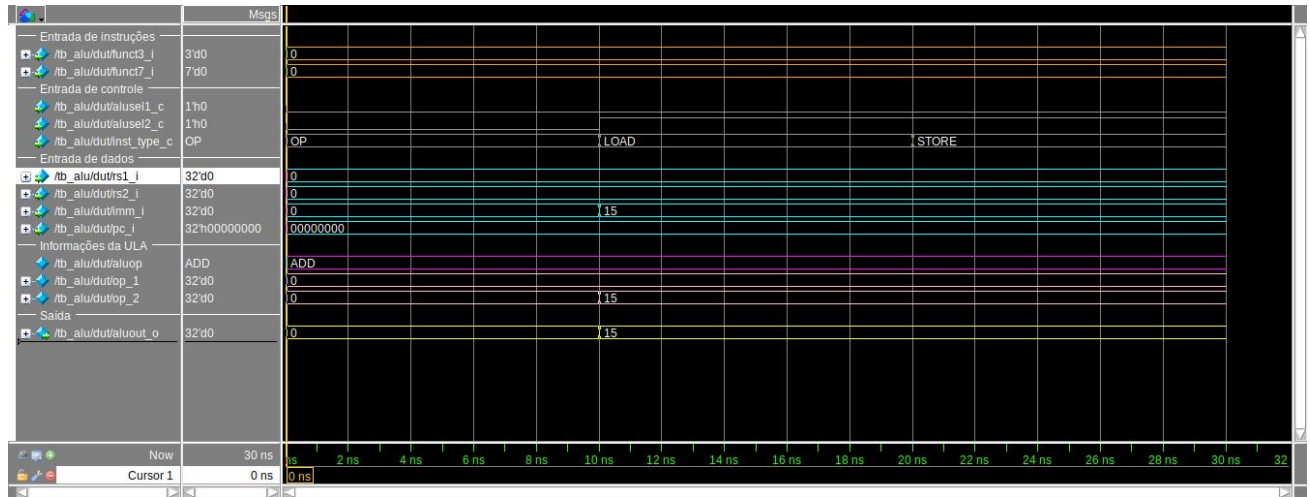
Para estas instruções, `inst_type_c` = LOAD (ou STORE), `aluse11_c` = 0, `aluse12_c` = 1, e `aluop` é fixado em ADD (pois o endereço efetivo é `rs1 + imm`). Na figura:

- `rs1_i` = 0x00000000, `imm_i` = 0x0000000F. A ALU realiza a soma: $0 + 15 = 15$. O sinal `aluout_o` torna-se **15** (decimal). Esse valor é o endereço calculado que será usado pela memória de dados.
- O testbench também testa o caso STORE (mesmo comportamento, pois a operação da ALU é idêntica: cálculo de endereço). A saída permanece consistente.

A ALU gera corretamente o endereço de acesso para as operações de carregamento e armazenamento, utilizando o modo de endereçamento por deslocamento.

4.5.6.4 Instruções de Desvio Condicional

Figura 22 – Waveform das instruções de desvio condicional (BEQ, BNE, BLT, BGE, BLTU, BGEU).



Fonte: O Autor

A Figura mostra a simulação das instruções de desvio. Neste modo, a ALU é configurada com `inst_type_c = BRANCH`, `alusel1_c = 0` e `alusel2_c = 0`, de modo que ambos os operandos são lidos do banco de registradores (`op_1 = rs1_i`, `op_2 = rs2_i`). O campo `funct3_i` determina a condição a ser avaliada. A ALU não produz um resultado aritmético completo, mas sim um **bit de condição** no LSB de `aluout_o` (os demais bits são zero). O waveform apresenta os seguintes casos:

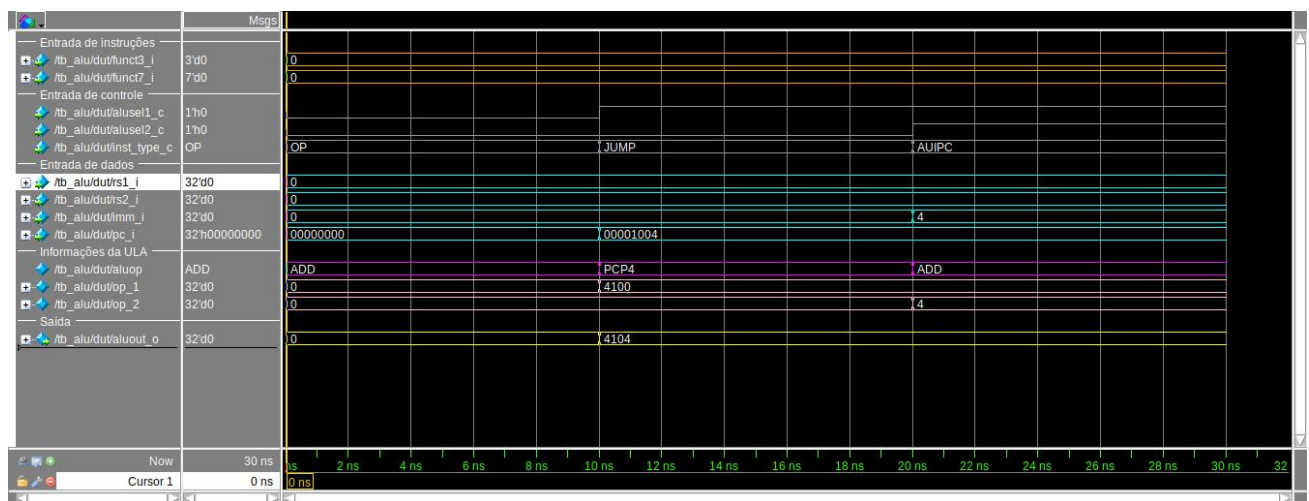
- **BEQ** (`funct3=0`): `rs1_i = 50`, `rs2_i = 50` condição verdadeira `aluout_o[0] = 1`. Em seguida, `rs2_i = 51` falso `aluout_o[0] = 0`.
- **BNE** (`funct3=1`): `rs1_i = 50`, `rs2_i = 50` falso (não é diferente) 0. `rs2_i = 51` verdadeiro 1.
- **BLT** (`funct3=4`): `rs1_i = 50`, `rs2_i = -50` `50 < -50?` Falso 0. `rs2_i = 51` `50 < 51?` Verdadeiro 1.
- **BGE** (`funct3=5`): `rs1_i = 50`, `rs2_i = -50` `50 >= -50?` Verdadeiro 1. `rs2_i = 51` `50 >= 51?` Falso 0.
- **BLTU** (`funct3=6`): `rs1_i = 50`, `rs2_i = 50` `50 < 50` (sem sinal) Falso 0. `rs2_i = 51` `50 < 51` Verdadeiro 1.
- **BGEU** (`funct3=7`): `rs1_i = 50`, `rs2_i = 50` `50 >= 50` Verdadeiro 1. `rs2_i = 51` `50 >= 51` Falso 0. `rs2_i = -51` (interpretado como sem sinal = `0xFFFFFCD`) `50 >= 0xFFFFFCD?` Falso 0.

Todos os bits de condição gerados pela ALU estão corretos de acordo com a especificação RISC-V. A saída `aluout_o` é subsequentemente utilizada pela unidade de controle para decidir se o PC deve ser atualizado com o endereço de desvio ou com `PC+4`.

A ALU avalia corretamente as condições para todos os desvios condicionais implementados, tanto para operandos com sinal quanto sem sinal.

4.5.6.5 Instruções de Salto (JAL, JALR) e AUIPC

Figura 23 – Waveform das instruções JAL, JALR e AUIPC.



Fonte: O Autor

- **JAL:** O testbench configura `inst_type_c = JUMP`, `alusel1_c = 1`, e `aluop = PCP4`. A entrada `pc_i = 0x00001004`. A saída `aluout_o` torna-se `pc_i + 4 = 0x00001008` (ou 4100 decimal, conforme a figura). Este é o endereço de retorno que será armazenado em `rd`.
- **JALR:** (não aparece explicitamente na figura, mas o testbench a inclui). A operação é semelhante, mas o cálculo do destino do salto é feito fora da ALU (no próprio módulo do processador). A ALU apenas fornece `pc+4` para o registrador de retorno.
- **AUIPC:** `inst_type_c = AUIPC`, `alusel1_c = 1`, `alusel2_c = 1` (soma PC + imediato). `pc_i = 0x00001004`, `imm_i = 4`. O imediato é deslocado 12 bits à esquerda na unidade de controle, mas a ALU apenas soma os valores recebidos. Pela figura, o resultado é 0x5004, indicando que o imediato foi tratado como 0x4000 ($4 \ll 12$) antes de chegar à ALU. Portanto, a ALU soma $0x1004 + 0x4000 = 0x5004$, que é o valor exibido. A saída está correta.

A ALU suporta as operações especiais PCP4 (para JAL/JALR) e a soma para AUIPC, gerando os valores esperados.

4.6 Banco de Registradores

O banco de registradores é um dos componentes centrais do processador, responsável por armazenar os 32 registradores de uso geral (x0 a x31), sendo que o registrador x0 é permanentemente ligado em zero, conforme a especificação RISC-V. O módulo foi implementado em SystemVerilog no arquivo `regfile.sv`.

4.6.1 Implementação do Módulo `regfile`

```

1  import data_structures::*;
2
3  module regfile (
4      input logic we_c, clk_c,
5      input data_t wdata_i,
6      input logic [4:0] rdaddr_i, rs1addr_i, rs2addr_i,
7      output register_t rs1_o, rs2_o
8  );
9      register_t x[31]; // 31 registradores (x1 a x31)
10
11     // Escrita sincrona (subida do clock)
12     always_ff @(posedge clk_c) begin : reg_write
13         if (we_c && rdaddr_i != 0)
14             x[rdaddr_i - 1] <= wdata_i;
15     end
16
17     // Leitura combinacional (com x0 fixo em 0)
18     assign rs1_o = (rs1addr_i == 0) ? 0 : x[rs1addr_i - 1];
19     assign rs2_o = (rs2addr_i == 0) ? 0 : x[rs2addr_i - 1];
20 endmodule

```

Listing 4.6 – Módulo `regfile.sv`

Observações sobre a implementação:

- O array `x` possui 31 elementos, correspondentes aos registradores `x1` a `x31`. O registrador `x0` é implementado exclusivamente pela lógica de leitura, que retorna zero quando o endereço é 0.
- A escrita ocorre apenas na borda de subida do clock (`posedge clk_c`) e quando a habilitação de escrita `we_c` está ativa e o endereço de destino é diferente de zero.
- O índice do array é calculado como `rdaddr_i - 1` (e similar para leitura), de modo que `rdaddr_i = 1` acessa `x[0]`, e `rdaddr_i = 31` acessa `x[30]`. Essa decisão de projeto economiza um elemento de memória, mas exige cuidado na indexação.
- As leituras são combinacionais, permitindo que os valores estejam disponíveis no mesmo ciclo em que os endereços são fornecidos, característica essencial para o processador monociclo.

4.6.2 Testbench do Banco de Registradores

O testbench `tb_regfile.sv` instancia o módulo `regfile` e aplica uma sequência de escrita e leitura em todos os registradores. O clock é gerado manualmente por meio de alternâncias de `clk_c` com atrasos de 10 unidades de tempo.

```
1 import data_structures::*;
2
3 module tb_regfile ();
4     logic clk_c, we_c;
5     data_t wdata_i;
6     logic [4:0] rdaddr_i, rs1addr_i, rs2addr_i;
7     register_t rs1_o, rs2_o;
8     data_t i;
9
10    regfile dut (.*);
11
12    initial begin
13        $display("Iniciando...");
14        we_c = 1;
15        clk_c = 0;
16        rs1addr_i = 0;
17        rs2addr_i = 0;
18        rdaddr_i = 0;
19
20        for (i = 0; i < 32; i += 1) begin
21            wdata_i = i * 4;
22            rdaddr_i = i;
23            rs1addr_i = i - 1;
24            rs2addr_i = i;
25            clk_c = 1;
26            #10;
27            clk_c = 0;
28            #10;
29        end
30
31        #10;
32        $stop;
33    end
34 endmodule
```

Listing 4.7 – Testbench `tb_regfile.sv`

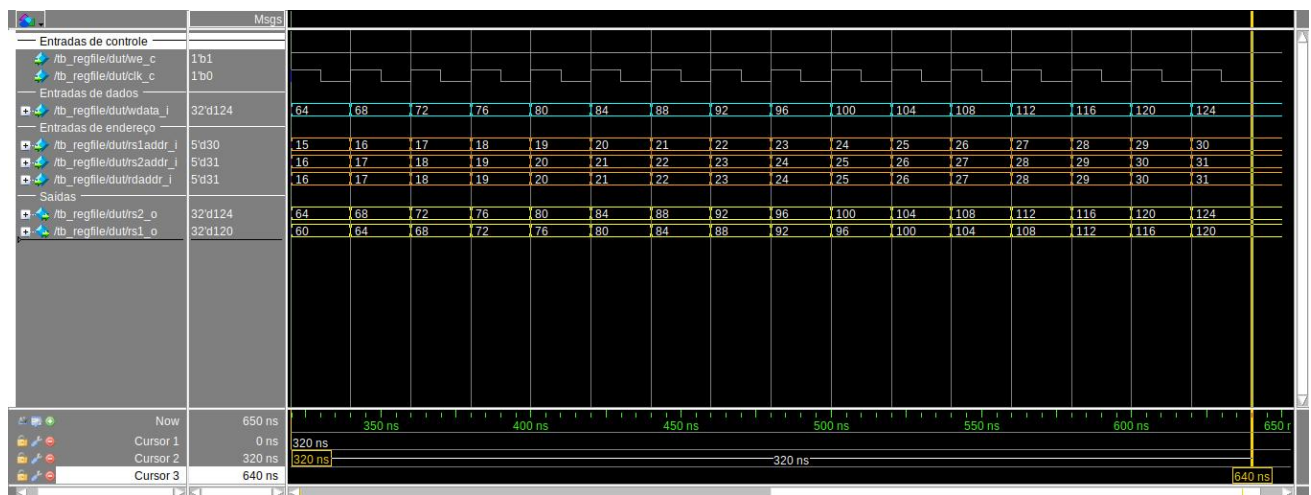
4.6.3 Resultados da Simulação

Figura 24 – Waveform do banco de registradores 1



Fonte: O Autor

Figura 25 – Waveform do banco de registradores 2



Fonte: O Autor

O testbench percorre todos os endereços de 0 a 31. Para cada valor de i , o dado escrito é $i * 4$. O sinal we_c permanece em 1 durante todo o teste. O clock clk_c alterna com período de 20 ns (10 ns em nível alto, 10 ns em nível baixo). Na borda de subida de cada ciclo, o registrador apontado por $rdaddr_i$ é atualizado com $wdata_i$. O banco de registradores implementa corretamente a lógica de leitura e escrita, respeitando a prioridade do registrador $x0$ sempre zero. A indexação baseada em $endereço - 1$ é funcional, embora fuja um pouco do estilo convencional. Para fins de integração com o restante do

processador, é importante garantir que os demais módulos (como a unidade de controle e a ALU) utilizem os mesmos endereços de registradores sem ajustes. Neste projeto, a interface com o banco de registradores segue o padrão RISC-V, sendo compatível.

4.7 Integração Banco de Registradores e ULA

A integração do banco de registradores com a ULA é um passo fundamental para a construção do caminho de dados do processador. Nesta montagem, as saídas de leitura do `regfile` (`rs1_o` e `rs2_o`) são conectadas diretamente às entradas `rs1_i` e `rs2_i` da ULA, enquanto o resultado da ULA (`aluout_o`) é realimentado na entrada de escrita de dados (`wdata_i`) do banco de registradores. Dessa forma, é possível executar uma sequência de instruções aritméticas e lógicas, escrevendo e lendo registradores a cada ciclo de clock.

4.7.1 Testbench de Integração

O testbench `tb.sv` instancia os dois módulos e os conecta conforme descrito. Além disso, gera um clock de período 10 ns (5 ns em nível alto, 5 ns em nível baixo) e aplica uma sequência de instruções que exercita o fluxo completo de leitura, processamento e escrita.

```

1  import data_structures::*;
2
3  module tb ();
4      // Sinais de controle, instrucao, dados e enderecos
5      logic clk_c, we_c, alusel1_c, alusel2_c;
6      inst_type_t inst_type_c;
7      funct3_t funct3_i;
8      funct7_t funct7_i;
9      data_t imm_i, pc_i;
10     logic [4:0] rdaddr_i, rs1addr_i, rs2addr_i;
11
12     register_t rs1, rs2, aluout;
13     data_t wdata; // Conectado a saida da ALU
14
15     // Instanciacao do banco de registradores
16     regfile regfile (
17         .clk_c(clk_c), .we_c(we_c), .wdata_i(wdata),
18         .rdaddr_i(rdaddr_i), .rs1addr_i(rs1addr_i), .rs2addr_i(rs2addr_i),
19         .rs1_o(rs1), .rs2_o(rs2)
20     );
21
22     // Instanciacao da ULA
23     alu alu (
24         .rs1_i(rs1), .rs2_i(rs2), .imm_i(imm_i), .pc_i(pc_i),
25         .funct3_i(funct3_i), .funct7_i(funct7_i),
26         .inst_type_c(inst_type_c),
27         .alusel1_c(alusel1_c), .alusel2_c(alusel2_c),
28         .aluout_o(aluout)
29     );
30
31     // Realimentacao: o resultado da ALU e o dado a ser escrito
32     assign wdata = aluout;
33

```

```

34 // Geracao de clock (periodo 10 ns, duty cycle 50%)
35 always #5 clk_c = !clk_c;
36
37 initial begin
38     // Inicializacoes
39     we_c = 0; clk_c = 0; alusel1_c = 0; alusel2_c = 0; inst_type_c = OP;
40     funct3_i = 0; funct7_i = 0; imm_i = 0; pc_i = 0;
41     rs1addr_i = 0; rs2addr_i = 0; rdaddr_i = 0;
42     #10;
43
44     // ADDI x1, x0, 99
45     inst_type_c = OPI; alusel2_c = 1; we_c = 1;
46     rdaddr_i = 1; rs1addr_i = 0; imm_i = 32'd99;
47     #10;
48
49     // ADDI x2, x1, 9
50     rdaddr_i = 2; rs1addr_i = 1; imm_i = 32'd9;
51     #10;
52
53     // ADDI x1, x0, 3
54     rs1addr_i = 0; rdaddr_i = 1; imm_i = 32'd3;
55     #10;
56
57     // DIV x3, x2, x1
58     alusel2_c = 0; inst_type_c = OP;
59     funct3_i = 7'd4; funct7_i = 7'd1;
60     rdaddr_i = 3; rs1addr_i = 2; rs2addr_i = 1;
61     #10;
62
63     $stop;
64 end
65 endmodule

```

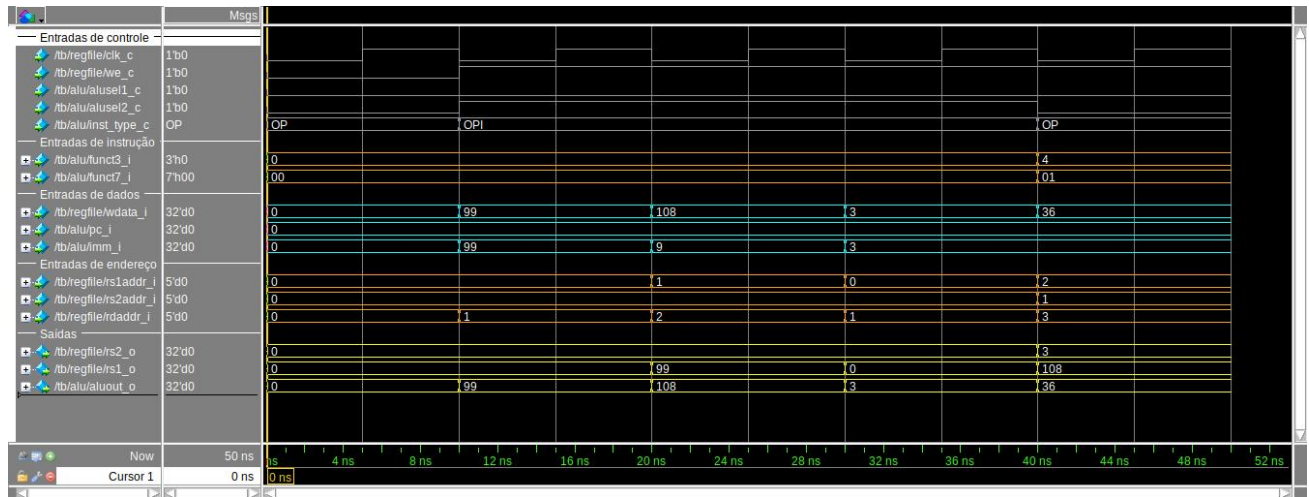
Listing 4.8 – Testbench de integração (tb.sv)

O testbench executa a seguinte sequência de operações (os valores imediatos foram escolhidos para produzir uma divisão inteira não trivial):

1. ADDI x1, x0, 99 escreve o valor 99 no registrador x1.
2. ADDI x2, x1, 9 lê x1 (99), soma 9 e escreve 108 em x2.
3. ADDI x1, x0, 3 escreve 3 em x1 (sobrescrevendo o valor anterior).
4. DIV x3, x2, x1 divide x2 (108) por x1 (3), resultando em 36, que é escrito em x3.

4.7.2 Resultados da Simulação Integrada

Figura 26 – Waveform da simulação integrada Banco de Registradores e ULA.



Fonte: O Autor

A Figura apresenta o comportamento dos sinais ao longo da simulação. A análise é dividida por intervalo de tempo, correspondente a cada instrução.

- **Ciclo 0:** Inicialização de variáveis
- **Ciclo 1 (escrita de 99 em x1):** `inst_type_c = OPI`, `alusel2_c = 1`, `we_c = 1`. `rs1addr_i = 0` leitura do registrador x0 (valor 0). `imm_i = 99`. A ULA realiza ADD e gera `aluout_o = 99`. Na borda de subida do clock, `rdaddr_i = 1` e `wdata_i = 99` x1 é atualizado para 99. As saídas `rs1_o` e `rs2_o` refletem os valores lidos (0 e 0, pois `rs2addr_i` ainda é 0).
- **Ciclo 2 (escrita de 108 em x2):** `rs1addr_i = 1` `rs1_o` passa a ser 99 (valor recémescrito). `imm_i = 9` ULA calcula $99 + 9 = 108$. `rdaddr_i = 2` na borda de clock, x2 recebe 108. Notase que a leitura de x1 ocorre no mesmo ciclo em que o valor 99 já está disponível, pois a escrita ocorreu no ciclo anterior.
- **Ciclo 3 (escrita de 3 em x1):** `rs1addr_i = 0` lê 0. `imm_i = 3` ULA gera 3. `rdaddr_i = 1` x1 é sobrescrito com 3. O waveform mostra que `rs1_o` (que antes era 99) se torna 3 apenas após a borda de clock, porque a leitura de `rs1` é combinacional e reflete o novo valor assim que a escrita termina? Na verdade, a escrita ocorre na borda de subida; durante o ciclo, a leitura para a próxima instrução (a DIV) utilizará o novo valor.
- **Ciclo 4 (divisão $108 / 3 = 36$):** `inst_type_c = OP`, `alusel2_c = 0` operandos provêm dos registradores. `rs1addr_i = 2` lê x2 (108). `rs2addr_i = 1` lê x1 (3).

`funct3 = 4, funct7 = 1` ULA decodifica `DIV` e calcula $108 / 3 = 36$. `rdaddr_i = 3` na borda de clock, `x3` recebe 36. As saídas `rs1_o` e `rs2_o` mostram 108 e 3, respectivamente, confirmando a leitura correta.

4.8 Memória de Instruções

A memória de instruções é um dos componentes fundamentais do processador, responsável por armazenar o programa a ser executado e fornecer a instrução correspondente ao endereço solicitado pelo Contador de Programa (PC). Neste projeto, a memória de instruções foi implementada como uma ROM (Read-Only Memory) de 1024 posições, cada uma contendo uma instrução de 32 bits. O conteúdo da memória é carregado a partir de um arquivo texto (`dados.txt`) no início da simulação.

4.8.1 Implementação do Módulo `inst_memory`

O módulo `inst_memory.sv` utiliza o package `data_structures` e possui uma interface simples: um sinal de reset (`RESET`), o endereço de leitura (`addr_i`) e a instrução de saída (`inst_o`).

```

1  import data_structures::*;
2
3  module inst_memory (
4      input wire RESET,
5      input address_t addr_i,
6      output instruction_t inst_o
7  );
8      instruction_t ins_mem[1024]; // 1024 palavras de 32 bits
9
10     // Inicializa a memoria com o conteudo do arquivo dados.txt
11     initial begin
12         $readmemh("dados.txt", ins_mem);
13     end
14
15     // Leitura combinacional: quando RESET = 1, a saida e o conteudo da memoria;
16     // quando RESET = 0, a saida e forçada para zero.
17     assign inst_o = (!RESET) ? 0 : ins_mem[addr_i[31:2]];
18 endmodule

```

Listing 4.9 – Módulo `inst_memory.sv`

Observações sobre a implementação:

- A memória é implementada como um array de 1024 elementos do tipo `instruction_t` (32 bits).
- A inicialização é feita através da tarefa `$readmemh`, que lê valores hexadecimais do arquivo `dados.txt` e os armazena sequencialmente a partir do endereço 0.

- O endereço de entrada `addr_i` é um vetor de 32 bits, mas como cada instrução ocupa 4 bytes, os dois bits menos significativos são ignorados (endereço word-aligned). Utiliza-se `addr_i[31:2]` como índice do array.
- O sinal `RESET` ativo em nível baixo (quando `RESET = 0`) zera a saída, forçando `inst_o = 0`. Quando `RESET = 1`, a memória opera normalmente. Essa lógica é útil para manter o processador em estado conhecido durante a inicialização.

4.8.2 Arquivo de Inicialização dados.txt

O arquivo `dados.txt` contém as instruções do programa de teste. Para esta simulação, foram utilizadas as seguintes instruções (em hexadecimal):

```
1 00500093
2 00A00113
3 002081B3
4 00302023
5 0000006F
```

Listing 4.10 – dados.txt programa de exemplo

Cada linha representa uma instrução de 32 bits no formato hexadecimal, armazenada consecutivamente a partir do endereço 0. A decodificação dessas instruções (de acordo com a ISA RISC-V) é apresentada na Tabela 9.

Tabela 9 – Programa de exemplo armazenado na memória de instruções.

Endereço	Instrução (hex)	Mnemônico
0x00	00500093	<code>addi x1, x0, 5</code>
0x04	00A00113	<code>addi x2, x0, 10</code>
0x08	002081B3	<code>add x3, x1, x2</code>
0x0C	00302023	<code>sw x3, 0(x0)</code>
0x10	0000006F	<code>jal x0, 0x10 (loop infinito)</code>

Fonte: O Autor

4.8.3 Testbench da Memória de Instruções

O testbench `tb.sv` instancia o módulo `inst_memory` e aplica uma sequência crescente de endereços para verificar a leitura correta das instruções armazenadas no arquivo `dados.txt`.

```
1 import data_structures::*;
2
3 module tb ();
4     logic RESET;
5     address_t addr_i;
6     instruction_t inst_o;
7     integer i;
8
```

```

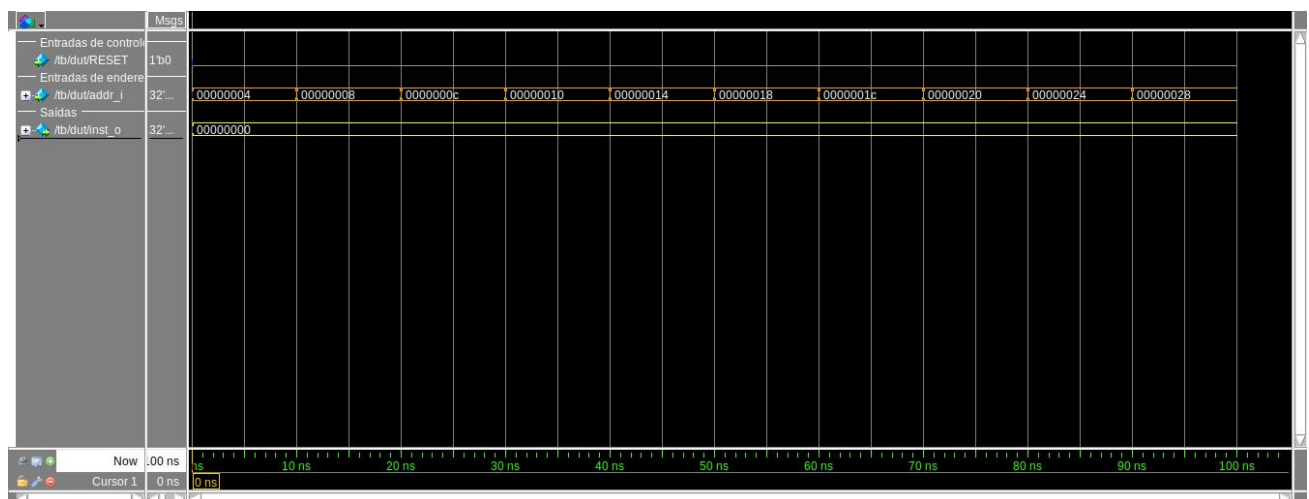
9    inst_memory dut (
10        .RESET(RESET),
11        .addr_i(addr_i),
12        .inst_o(inst_o)
13    );
14
15    initial begin
16        $display("Comecando TB...");
17        RESET = 1;          // RESET inativo (memoria habilitada), para o teste inativo basta
                             // modificar de forma que RESET = 0
18        addr_i = 0;
19
20        for (i = 0; i < 10; i++) begin
21            addr_i += 4;      // incrementa de 4 em 4 bytes
22            #10;
23        end
24
25        $stop;
26    end
27 endmodule

```

Listing 4.11 – Testbench tb.sv para a memória de instruções

4.8.4 Resultados da Simulação

Dois cenários de simulação foram avaliados: com o sinal **RESET** ativo (forçando a saída a zero) e com o **RESET** inativo (leitura normal da memória). Os waveforms correspondentes são apresentados nas Figuras a seguir:

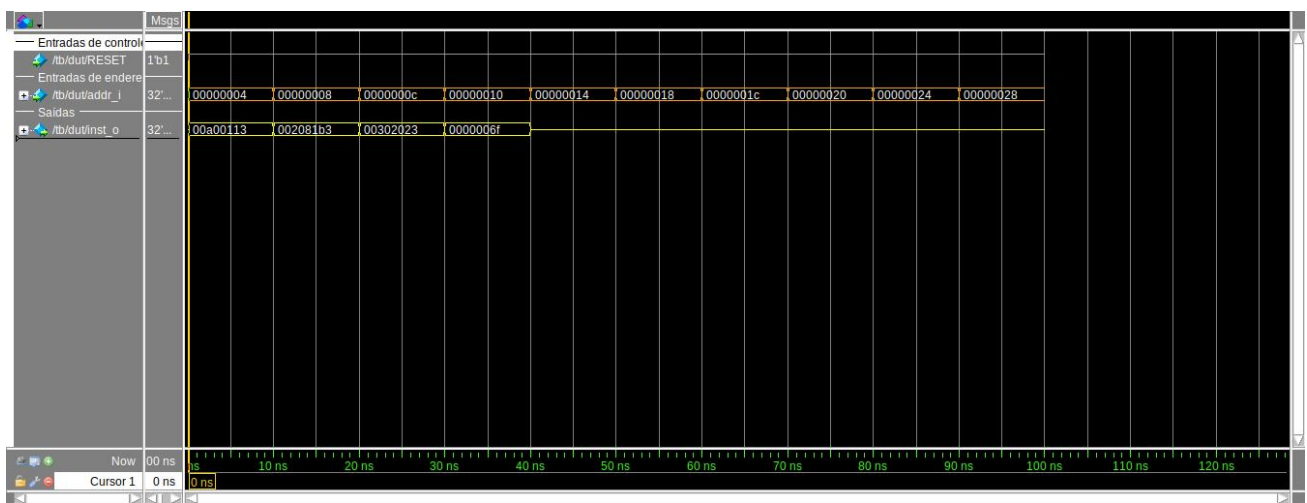
Figura 27 – Waveform da memória de instruções com **RESET** ativo (**RESET** = 0).

Fonte: O Autor

Neste caso, o sinal **RESET** está ativo (nível baixo), o que força a saída a zero independentemente do endereço.

- **RESET** = 0 durante toda a simulação.
- O testbench aplica a mesma sequência de endereços (0x00, 0x04, 0x08,).
- A saída `inst_o` permanece **0x00000000** para todos os endereços, conforme a lógica `assign inst_o = (!RESET) ? 0 : ...`. Esse comportamento é desejável durante a fase de reset do processador, garantindo que nenhuma instrução inválida seja executada.

Figura 28 – Waveform da memória de instruções com RESET inativo (RESET = 1).



Fonte: O Autor

Nesta configuração, a memória está habilitada e deve apresentar na saída a instrução correspondente ao endereço solicitado.

- **RESET** = 1 durante toda a simulação (sinal em nível alto).
- O endereço `addr_i` inicia em 0x00000000 e é incrementado de 4 em 4 a cada 10 ns (valores: 0x00, 0x04, 0x08, 0x0C, 0x10, 0x14, 0x18, 0x1C, 0x20, 0x24,).
- A saída `inst_o` assume os valores armazenados no arquivo `dados.txt` para os endereços correspondentes:
 - `addr_i = 0x00 inst_o = 0x00500093` (`addi x1, x0, 5`).
 - `addr_i = 0x04 inst_o = 0x00A00113` (`addi x2, x0, 10`).
 - `addr_i = 0x08 inst_o = 0x002081B3` (`add x3, x1, x2`).
 - `addr_i = 0x0C inst_o = 0x00302023` (`sw x3, 0(x0)`).
 - `addr_i = 0x10 inst_o = 0x0000006F` (`jal x0, 0x10`).

- Endereços além de 0x10 (como 0x14, 0x18,) não foram inicializados no arquivo, portanto o conteúdo da memória é indefinido (os waveforms mostram valores como 32'hxxxxxx ou zeros, dependendo da ferramenta de simulação).
- A saída responde de forma combinacional às mudanças no endereço, sem atrasos visíveis.

4.9 Memória de Dados

A memória de dados é responsável por armazenar valores manipulados pelo programa durante a execução, suportando operações de leitura (**load**) e escrita (**store**) com diferentes granularidades: byte (8 bits), meia palavra (16 bits) e palavra (32 bits). O módulo foi implementado em SystemVerilog no arquivo `data_memory.sv`, utilizando uma RAM síncrona de 1024 posições de 32 bits.

4.9.1 Implementação do Módulo

O módulo possui interface síncrona (escrita e leitura ocorrem na borda de subida do clock) e inclui uma lógica de *byte enable* para permitir acessos parciais à memória, conforme exigido pelas instruções **lb**, **lbu**, **lw**, **lh**, **lhu**, **sb**, **sh** e **sw**.

```

1  import data_structures::*;
2
3  module data_memory (
4      input logic clk_c, memwrite_c, memread_c,
5      input funct3_t funct3_i,
6      input address_t addr_i,
7      input data_t wdata_i,
8      output data_t rdata_o
9  );
10     data_t data_mem[1024];    // 1024 palavras de 32 bits
11
12     // Geracao do byte enable com base no funct3 e nos bits baixos do endereco
13     logic [3:0] be;
14     always_comb begin : be_generator
15         if (funct3_i == 3'b000 || funct3_i == 3'b100) begin // lb ou lbu
16             case (addr_i[1:0])
17                 2'b00: be = 4'b0001;
18                 2'b01: be = 4'b0010;
19                 2'b10: be = 4'b0100;
20                 2'b11: be = 4'b1000;
21                 default: be = 4'b0000;
22             endcase
23         end else if (funct3_i == 3'b001 || funct3_i == 3'b101) begin // lh ou lhu
24             case (addr_i[1])
25                 1'b0: be = 4'b0011;
26                 1'b1: be = 4'b1100;
27                 default: be = 4'b0000;
28             endcase
29         end else if (funct3_i == 3'b010) begin // lw ou sw
30             be = 4'b1111;

```



```

31     end else begin
32         be = 4'b0000;
33     end
34 end
35
36 // Escrita e leitura sincronas
37 always_ff @(posedge clk_c) begin
38     // Escrita
39     if (memwrite_c) begin
40         case (be)
41             4'b0001: data_mem[addr_i[31:2]][7:0]    <= wdata_i[7:0];
42             4'b0010: data_mem[addr_i[31:2]][15:8]   <= wdata_i[7:0];
43             4'b0100: data_mem[addr_i[31:2]][23:16]  <= wdata_i[7:0];
44             4'b1000: data_mem[addr_i[31:2]][31:24]  <= wdata_i[7:0];
45             4'b0011: data_mem[addr_i[31:2]][15:0]   <= wdata_i[15:0];
46             4'b1100: data_mem[addr_i[31:2]][31:16]  <= wdata_i[15:0];
47             4'b1111: data_mem[addr_i[31:2]][31:0]   <= wdata_i[31:0];
48             default: ;
49         endcase
50     end
51
52     // Leitura
53     if (memread_c) begin
54         if (funct3_i == 3'b000 || funct3_i == 3'b001 || funct3_i == 3'b010) begin
55             // Leitura com extensao de sinal (lb, lh, lw)
56             case (be)
57                 4'b0001: rdata_o <= 32'(signed'(data_mem[addr_i[31:2]][7:0]));
58                 4'b0010: rdata_o <= 32'(signed'(data_mem[addr_i[31:2]][15:8]));
59                 4'b0100: rdata_o <= 32'(signed'(data_mem[addr_i[31:2]][23:16]));
60                 4'b1000: rdata_o <= 32'(signed'(data_mem[addr_i[31:2]][31:24]));
61                 4'b0011: rdata_o <= 32'(signed'(data_mem[addr_i[31:2]][15:0]));
62                 4'b1100: rdata_o <= 32'(signed'(data_mem[addr_i[31:2]][31:16]));
63                 4'b1111: rdata_o <= data_mem[addr_i[31:2]][31:0];
64                 default: ;
65             endcase
66         end
67         if (funct3_i == 3'b100 || funct3_i == 3'b101) begin
68             // Leitura com extensao de zero (lbu, lhu)
69             case (be)
70                 4'b0001: rdata_o <= 32'(unsigned'(data_mem[addr_i[31:2]][7:0]));
71                 4'b0010: rdata_o <= 32'(unsigned'(data_mem[addr_i[31:2]][15:8]));
72                 4'b0100: rdata_o <= 32'(unsigned'(data_mem[addr_i[31:2]][23:16]));
73                 4'b1000: rdata_o <= 32'(unsigned'(data_mem[addr_i[31:2]][31:24]));
74                 4'b0011: rdata_o <= 32'(unsigned'(data_mem[addr_i[31:2]][15:0]));
75                 4'b1100: rdata_o <= 32'(unsigned'(data_mem[addr_i[31:2]][31:16]));
76                 default: ;
77             endcase
78         end
79     end
80 end
81 endmodule

```

Listing 4.12 – Módulo data_memory.sv

Características da implementação:

- A memória é endereçada por palavra (word-aligned): `addr_i[31:2]` seleciona a palavra, e os bits `addr_i[1:0]` determinam o deslocamento dentro da palavra (para

acessos a byte/meia palavra).

- O sinal **be** (byte enable) é gerado combinacionalmente a partir de **funct3_i** e dos bits baixos do endereço, indicando quais bytes da palavra serão lidos/escritos.
- As operações de escrita e leitura ocorrem na borda de subida do clock (**posedge clk_c**), caracterizando uma memória RAM síncrona.
- Para leituras, o módulo distingue entre extensão de sinal (**1b, 1h**) e extensão de zero (**1bu, 1hu**), convertendo o valor lido para 32 bits conforme a especificação RISC-V.
- A memória não é inicializada automaticamente (os valores iniciais são **x** indeterminados), mas pode ser inicializada via **\$readmemh** se desejado.

4.9.2 Testbench da Memória de Dados

O testbench **tb.sv** instancia o módulo **data_memory**, gera um clock de período 10 ns e aplica uma sequência de escrita e leitura para verificar os acessos em diferentes granularidades.

```

1  import data_structures::*;
2
3  module tb();
4      logic clk_c, memwrite_c, memread_c;
5      funct3_t funct3_i;
6      address_t addr_i;
7      data_t wdata_i, rdata_o;
8
9      data_memory dut (.*); // Conexao automatica
10
11     always #5 clk_c = !clk_c;
12
13     initial begin
14         $display("Iniciando TB memoria de dados...");
15         clk_c = 0; memwrite_c = 0; memread_c = 0;
16         funct3_i = 2; // word
17         addr_i = 0; wdata_i = 0;
18         #10;
19
20         // Escrita de palavra (2048) no endereco 0
21         memwrite_c = 1;
22         wdata_i = 2048;
23         #10;
24
25         // Leitura da palavra no endereco 0
26         memwrite_c = 0;
27         memread_c = 1;
28         #10;
29
30         // Escrita de byte (10) no endereco 0 (funcao lb/sb)
31         memwrite_c = 1; memread_c = 0;
32         funct3_i = 0; // byte
33         wdata_i = 10; addr_i = 0;
34         #10;

```

```

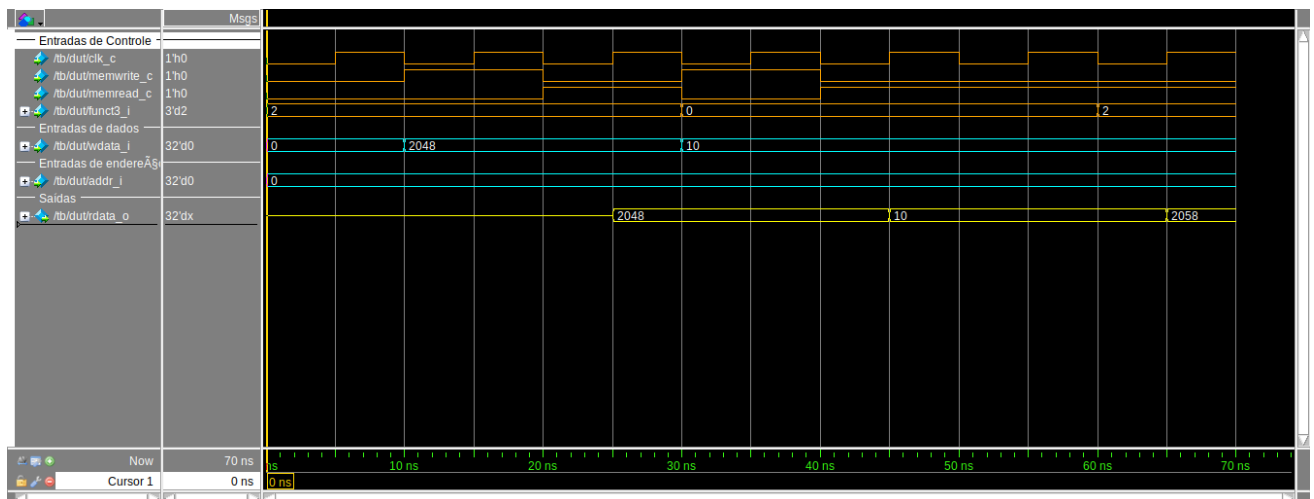
35
36 // Leitura do byte no endereço 0
37 memwrite_c = 0; memread_c = 1;
38 #10;
39
40 addr_i = 0; // mantém o endereço
41 #10;
42 funct3_i = 2; // volta para word, leitura do mesmo endereço
43 #10;
44
45 $stop;
46 end
47 endmodule

```

Listing 4.13 – Testbench tb.sv para a memória de dados

4.9.3 Resultados da Simulação

Figura 29 – Waveform da simulação da memória de dados.



Fonte: O Autor

A análise é dividida nos intervalos de tempo (em ns) conforme a sequência aplicada.

- **t = 0 a 10 ns (inicialização):** clk_c alterna a cada 5 ns. Inicialmente, memwrite_c = 0, memread_c = 0, funct3_i = 2 (word), addr_i = 0, wdata_i = 0. A saída rdata_o está em alta impedância (32'hx) porque não há leitura ativa.
- **t = 10 a 20 ns (escrita da palavra 2048):** memwrite_c = 1, wdata_i = 2048. Na borda de subida do clock em t = 15 ns (aproximadamente), o valor 2048 é escrito na palavra de endereço 0 (os primeiros 4 bytes da memória). O sinal rdata_o permanece indeterminado, pois não há leitura simultânea.
- **t = 20 a 30 ns (leitura da palavra):** memwrite_c = 0, memread_c = 1, funct3_i = 2. Na borda de subida do clock em t = 25 ns, a memória realiza a leitura do ende-

reço 0 e coloca o valor lido (2048) em `rdata_o`. O waveform deve mostrar `rdata_o = 2048` a partir desse momento (até a próxima leitura).

- **t = 30 a 40 ns (escrita de byte 10 no endereço 0):** `memwrite_c = 1`, `memread_c = 0`, `funct3_i = 0` (byte), `wdata_i = 10`, `addr_i = 0`. Como `addr_i[1:0] = 0`, o *byte enable* `be` gerado é `4'b0001` (apenas o byte menos significativo). Na borda de clock ($t = 35$ ns), o byte 10 é escrito no primeiro byte da palavra no endereço 0, preservando os outros três bytes (que ainda contêm $2048 = 0x00000800$). Após essa operação, o conteúdo da palavra passa a ser: `0x0000080A` (assumindo little-endian, o byte 10 no endereço 0).
- **t = 40 a 50 ns (leitura do byte):** `memwrite_c = 0`, `memread_c = 1`, `funct3_i = 0` (leitura de byte com extensão de sinal). Na borda de clock ($t = 45$ ns), a memória lê o byte do endereço 0, estende o sinal (como o valor 10 é positivo, extensão resulta em 10) e coloca `rdata_o = 10`. O waveform deve mostrar a saída igual a 10.
- **t = 50 a 60 ns (endereço mantido):** O testbench mantém `addr_i = 0` sem novas operações. A saída permanece em 10 (último valor lido).
- **t = 60 a 70 ns (leitura como palavra do mesmo endereço):** `funct3_i = 2` (leitura de palavra). Na borda de clock ($t = 65$ ns), a memória lê a palavra completa do endereço 0, que agora contém `0x0000080A = 2058` ($2048 + 10$). O waveform deve mostrar `rdata_o = 2058`, confirmando que a escrita parcial (byte) não afetou os demais bytes.

4.10 Unidade de Branch

A unidade de branch é responsável por calcular o próximo endereço do Contador de Programa (PC) durante a execução de instruções de desvio condicional (`beq`, `bne`, etc.), saltos incondicionais (`jal`) e saltos indiretos (`jalr`). Este módulo recebe os sinais de controle gerados pela unidade de controle (`jump_c`, `branch_c`, `pcsource_c`), o valor da ALU (que fornece o bit de condição para os desvios), os valores dos operandos (`pc_i`, `imm_i`, `rs1_i`) e produz o próximo PC (`pc_next_o`).

4.10.1 Implementação do Módulo `branch_unit`

O módulo `branch_unit.sv` utiliza lógica combinacional para calcular o próximo PC de acordo com a equação:

$$pc_next = base + incremento$$

onde a base é selecionada pelo sinal `pcsource_c` (`rs1_i` para JALR, `pc_i` para os demais casos), e o incremento é `imm_i` para saltos ou desvios ativos, caso contrário 4 (incremento normal).

```

1  import data_structures::*;
2
3  module branch_unit (
4      input register_t rs1_i, pc_i,
5      input data_t aluout_i, imm_i,
6      input logic jump_c, branch_c, pcsource_c,
7      output register_t pc_next_o
8  );
9      data_t sum1 = 0, sum2 = 0;
10
11     always_comb begin : branch_unit
12         sum1 = (pcsource_c) ? rs1_i : pc_i;
13         sum2 = (jump_c || (branch_c && aluout_i[0])) ? imm_i : 32'(4);
14         pc_next_o = sum1 + sum2;
15     end
16 endmodule

```

Listing 4.14 – Módulo `branch_unit.sv`

Funcionamento:

- `pcsource_c`: quando em 1, a base do endereço é `rs1_i` (usado em `jalr`); quando em 0, a base é o PC atual.
- `sum2`: é `imm_i` se houver salto incondicional (`jump_c = 1`) ou um desvio condicional ativo (`branch_c = 1` e o bit de condição `aluout_i[0] = 1`); caso contrário, `sum2 = 4` para avançar para a próxima instrução sequencial.
- A soma `sum1 + sum2` produz o endereço da próxima instrução.

4.10.2 Testbench da Unidade de Branch

O testbench `branch_tb.sv` aplica estímulos para verificar as três operações principais: desvio condicional (`branch`), salto e ligação (`jal`) e salto indireto (`jalr`). O clock não é necessário, pois o módulo é combinacional.

```

1  import data_structures::*;
2
3  module branch_tb ();
4      register_t rs1_i, pc_i, pc_next_o;
5      data_t aluout_i, imm_i;
6      logic jump_c, branch_c, pcsource_c;
7
8      branch_unit dut (.*);
9
10     initial begin
11         $display("Inicializando valores...");
12         rs1_i = 0; pc_i = 0; imm_i = 0; aluout_i = 0;
13         jump_c = 0; branch_c = 0; pcsource_c = 0;
14         #10;

```

```

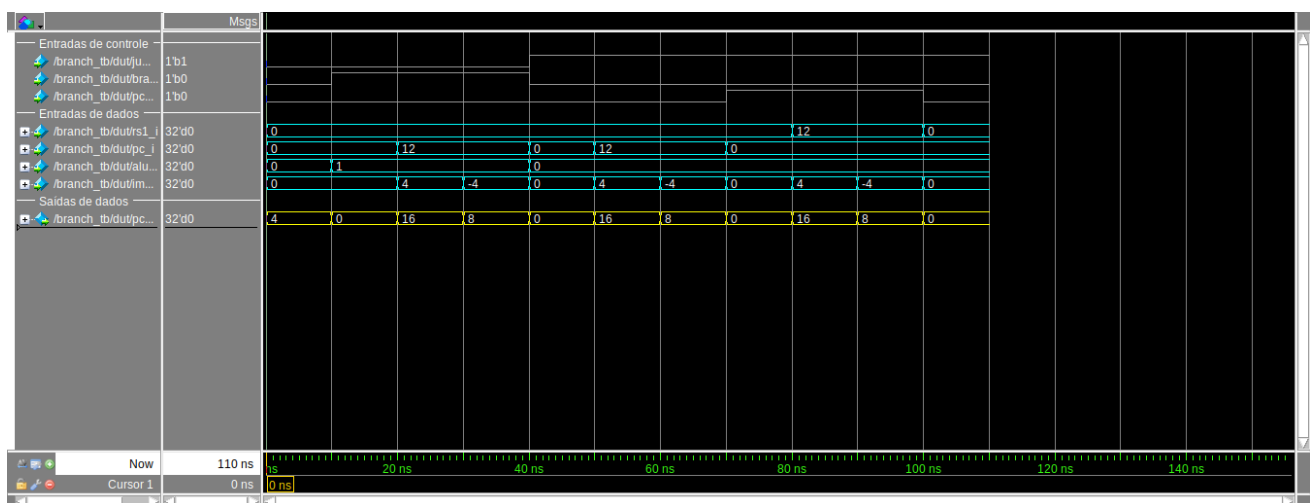
15
16 // Teste BRANCH: branch_c=1, condicao verdadeira
17 branch_c = 1; aluout_i[0] = 1;
18 pc_i = 12; imm_i = 4; // esperado 16
19 #10;
20
21 pc_i = 12; imm_i = -4; // esperado 8
22 #10;
23
24 // Teste JAL: jump_c=1
25 branch_c = 0; jump_c = 1;
26 pc_i = 12; imm_i = 4; // esperado 16
27 #10;
28
29 pc_i = 12; imm_i = -4; // esperado 8
30 #10;
31
32 // Teste JALR: pcsource_c=1
33 jump_c = 0; pcsource_c = 1;
34 rs1_i = 12; imm_i = 4; // esperado 16
35 #10;
36
37 rs1_i = 12; imm_i = -4; // esperado 8
38 #10;
39
40 $stop;
41 end
42 endmodule

```

Listing 4.15 – Testbench branch_tb.sv

4.10.3 Resultados da Simulação

Figura 30 – Waveform da simulação da unidade de branch.



Fonte: O Autor

A Figura 30 mostra os resultados dos testes. A análise é dividida pelos três grupos de operação:

- **Desvio condicional (branch):** `branch_c = 1`, `aluout_i[0] = 1` (condição verdadeira). Para `pc_i = 12`, `imm_i = 4` `pc_next_o = 16` (correto). Para `pc_i = 12`, `imm_i = -4` `pc_next_o = 8` (correto).
- **Salto e ligação (jal):** `jump_c = 1`, independente da condição. `pc_i = 12`, `imm_i = 4` `pc_next_o = 16`. `pc_i = 12`, `imm_i = -4` `pc_next_o = 8`. O módulo ignora `aluout_i` e `branch_c` nesta configuração.
- **Salto indireto (jalr):** `pcsource_c = 1`, base é `rs1_i` em vez de `pc_i`. `rs1_i = 12`, `imm_i = 4` `pc_next_o = 16`. `rs1_i = 12`, `imm_i = -4` `pc_next_o = 8`. O comportamento está de acordo com a especificação RISC-V para a instrução `jalr`.

Todos os resultados observados no waveform estão em conformidade com os valores esperados. A unidade de branch opera de forma combinacional, produzindo o próximo endereço sem atrasos adicionais, o que é essencial para o funcionamento do processador monociclo.

4.11 Contador de Programa

O Contador de Programa (Program Counter PC) é o registrador que armazena o endereço da instrução corrente. A cada ciclo de clock, ele é atualizado com o próximo endereço fornecido pela unidade de branch. Neste projeto, o PC não foi implementado como um módulo separado, mas sim como um bloco interno no código de integração final (topo do processador), sendo assim, sua waveform não foi gerada, a corretude do PC será avaliada na simulação da integração. A lógica de atualização é a seguinte:

```

1 always_ff @(posedge clk_c) begin : pc_update
2     if (RESET)
3         pc <= 32'd0;
4     else
5         pc <= pc_next;
6 end

```

Listing 4.16 – Lógica do PC (dentro do módulo topo)

onde:

- `clk_c` é o sinal de clock global do processador.
- `RESET` é um sinal de reset síncrono (quando ativo, o PC é zerado na próxima borda de clock).
- `pc_next` é o próximo endereço calculado pela unidade de branch (`branch_unit`).
- `pc` é o registrador interno que armazena o endereço atual, cujo valor é utilizado para acessar a memória de instruções.

A saída do PC é conectada diretamente à entrada de endereço da memória de instruções, permitindo a busca da instrução correspondente a cada ciclo.

4.12 Gerador de Imediatos

O gerador de immediatos é um módulo combinacional responsável por extrair e estender o valor imediato presente em diferentes campos da instrução, conforme o formato definido pela arquitetura RISC-V. Cada tipo de instrução (I, S, B, U, J) possui uma disposição distinta dos bits do imediato dentro da palavra de 32 bits. Este módulo recebe a instrução completa e o tipo de instrução decodificado, produzindo o valor imediato de 32 bits (já com extensão de sinal, quando necessário).

4.12.1 Implementação do Módulo `immediate_generator`

O módulo `immediate_generator.sv` implementa a lógica de extração para os seguintes tipos:

- **OPI e LOAD:** imediato de 12 bits nos campos `inst_i[31:20]`, com extensão de sinal.
- **AUIPC:** imediato de 20 bits nos campos `inst_i[31:12]`, deslocado à esquerda por 12 bits (preenchendo os bits inferiores com zeros).
- **STORE:** imediato de 12 bits composto por `inst_i[31:25]` (bits altos) e `inst_i[11:7]` (bits baixos), com extensão de sinal.
- **BRANCH:** imediato de 13 bits (incluindo o bit de deslocamento 0 no LSB), formado por `inst_i[31]`, `inst_i[7]`, `inst_i[30:25]`, `inst_i[11:8]`, e o bit 0 fixo em 0. O valor é estendido com sinal.
- **JUMP:** imediato de 21 bits (incluindo o bit 0 no LSB), formado por `inst_i[31]`, `inst_i[19:12]`, `inst_i[20]`, `inst_i[30:21]`, e o bit 0 fixo em 0. O valor é estendido com sinal.

O código a seguir implementa essa lógica.

```

1  import data_structures::*;
2
3  module immediate_generator (
4      input instruction_t inst_i,
5      input inst_type_t   inst_type_c,
6      output data_t imm_o
7  );
8      always_comb begin : immediate_generator
9          imm_o = 0;
10         case (inst_type_c)

```



```

11     OPI, LOAD:
12         imm_o = 32'(signed'(inst_i[31:20]));
13     AUIPC:
14         imm_o = {inst_i[31:12], 12'b00000000000000};
15     STORE:
16         imm_o = 32'(signed'({inst_i[31:25], inst_i[11:7]}));
17     BRANCH:
18         imm_o = 32'(signed'({inst_i[31], inst_i[7], inst_i[30:25], inst_i[11:8], 1'b0}));
19     JUMP:
20         imm_o = 32'(signed'({inst_i[31], inst_i[19:12], inst_i[20], inst_i[30:21], 1'b0}
21         ));
22     default: ;
23 endcase
24 end
25 endmodule

```

Listing 4.17 – Módulo immediate_generator.sv

4.12.2 Testbench do Gerador de Imediatos

O testbench `immgen_tb.sv` aplica valores de instrução para cada tipo e verifica o imediato gerado.

```

1  import data_structures::*;
2
3  module immgen_tb ();
4      instruction_t inst_i;
5      inst_type_t inst_type_c;
6      data_t imm_o;
7
8      immediate_generator dut (.*);
9
10     initial begin
11         $display("Iniciando valores...");
12         inst_i = 0; inst_type_c = OP;
13         #10;
14
15         // Testes para OPI e LOAD (imediatos de 12 bits)
16         inst_type_c = OPI;
17         inst_i = 32'b11111111111100000000000000000000; // esperado -1
18         #10;
19         inst_i = 32'b00000000000010000000000000000000; // esperado 1
20         #10;
21
22         // Testes para AUIPC (imediato superior)
23         inst_type_c = AUIPC;
24         inst_i = 32'b01000000000000000000000000000000; // esperado 0x40000000 (1073741824)
25         #10;
26         inst_i = 32'b11000000000000000000000000000000; // esperado -1073741824
27         #10;
28
29         // Testes para STORE
30         inst_type_c = STORE;
31         inst_i = 32'b011111100000000000000000111110000000; // esperado 2047
32         #10;
33         inst_i = 32'b100000000000100010010000010100011; // esperado -2047
34         #10;
35

```

```

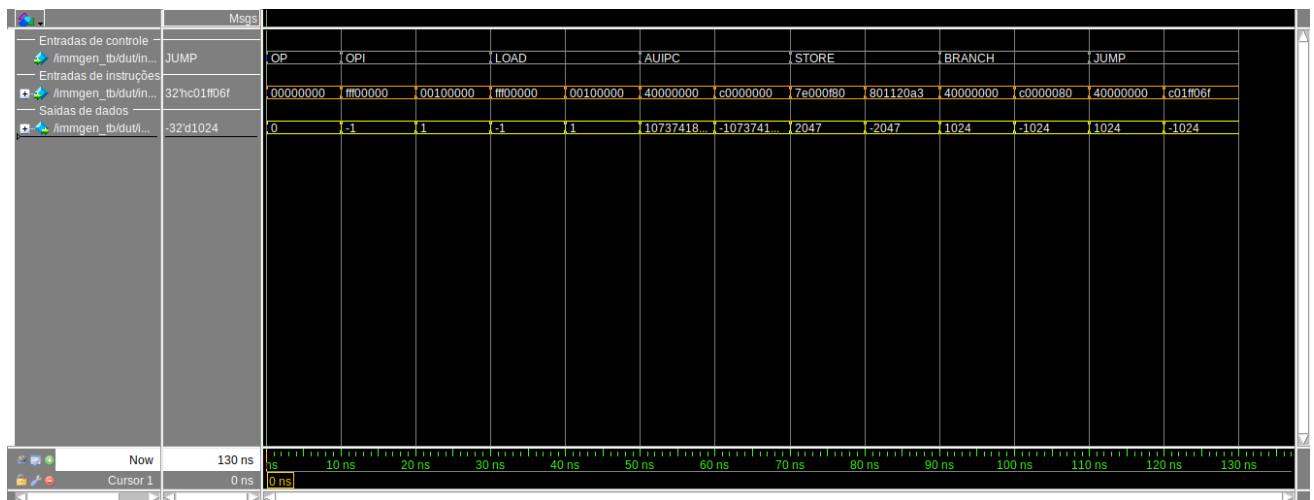
36 // Testes para BRANCH
37 inst_type_c = BRANCH;
38 inst_i = 32'b01000000000000000000000000000000; // esperado 1024
39 #10;
40 inst_i = 32'b110000000000000000000000010000000; // esperado -1024
41 #10;
42
43 // Testes para JUMP
44 inst_type_c = JUMP;
45 inst_i = 32'b01000000000000000000000000000000; // esperado 1024
46 #10;
47 inst_i = 32'b11000000000111111111000001101111; // esperado -1024
48 #10;
49
50 $stop;
51 end
52 endmodule

```

Listing 4.18 – Testbench immgen_tb.sv

4.12.3 Resultados da Simulação

Figura 31 – Waveform da simulação do gerador de imediatos.



Fonte: O Autor

A Figura 31 apresenta os resultados dos testes. A análise é feita por tipo de instrução:

- **OPI / LOAD:** Para `inst_type_c = OPI`, com `inst_i = 0xFFFFF000` (em 32 bits: `32'b11111111111111000000000000000000`) imediato = -1. Em seguida, `inst_i = 0x00100000` imediato = 1. Os valores estão corretos e a extensão de sinal é aplicada.
- **AUIPC:** `inst_type_c = AUIPC`, `inst_i = 0x40000000` imediato = `0x40000000` (1073741824). `inst_i = 0xC0000000` imediato = -1073741824. O imediato é extraído dos bits 31:12 e deslocado à esquerda, conforme esperado.

- **STORE:** `inst_type_c = STORE, inst_i = 0x7E000F80` (32'b011111110000000000000111110000) imediato = 2047. `inst_i = 0x801120A3` (aproximadamente) imediato = -2047. A concatenação dos campos `inst_i[31:25]` e `inst_i[11:7]` e a extensão de sinal estão corretas.
- **BRANCH:** `inst_type_c = BRANCH, inst_i = 0x40000000` imediato = 1024. `inst_i = 0xC0000080` imediato = -1024. Observa-se que o bit LSB é fixado em 0, e a concatenação segue o formato B.
- **JUMP:** `inst_type_c = JUMP, inst_i = 0x40000000` imediato = 1024. `inst_i = 0xC01FF06F` (instrução jal) imediato = -1024. O formato J é corretamente reconstruído, incluindo o bit de sinal e o bit 0 fixo.

Todos os resultados observados no waveform correspondem aos valores esperados. O gerador de immediatos opera de forma combinacional e sem atrasos, sendo essencial para fornecer o valor imediato correto à ALU e à unidade de branch.

5 Considerações Finais

O trabalho desenvolvido até este ponto atingiu os objetivos relacionados à implementação e validação dos componentes centrais da unidade de processamento de um processador RISC-V de 32 bits. Com base nos arquivos codificados em SystemVerilog e nas simulações realizadas, os seguintes elementos foram completamente projetados, testados e documentados:

- **Unidade Lógica e Aritmética (ULA):** implementada com suporte a 24 operações, incluindo aritméticas básicas, lógicas, deslocamentos, comparações e a extensão de multiplicação e divisão (RV32M). O módulo foi validado por meio de testbenches exaustivos, e as formas de onda comprovam a correção de cada operação.
- **Banco de Registradores (Register File):** composto por 32 registradores de 32 bits, com o registrador `x0` permanentemente ligado em zero. As simulações demonstraram a correta escrita síncrona e leitura combinacional.
- **Integração ULA e Banco de Registradores:** o testbench conjunto executou uma sequência de instruções (somas com imediato e uma divisão), verificando o fluxo completo de leitura, processamento e escrita dos resultados.
- **Memória de Instruções (ROM):** capaz de carregar um programa a partir de um arquivo texto (`dados.txt`) e fornecer a instrução correta para cada endereço, com tratamento do sinal de RESET.
- **Memória de Dados (RAM):** implementada como memória síncrona de 1024 palavras, com lógica de *byte enable* que permite acessos a byte, meia palavra e palavra, além de distinguir corretamente extensão de sinal (`1b`, `1h`) de extensão de zero (`1bu`, `1hu`). As simulações confirmaram escritas parciais e leituras com as diferentes granularidades.
- **Gerador de Imediatos (Immediate Generator):** módulo combinacional que extrai e estende os imediatos dos formatos I, S, B, U e J, validado com testbench específico e formas de onda.
- **Unidade de Branch:** módulo combinacional que calcula o próximo PC para desvios condicionais, `jal` e `jalr`, validado com testbench próprio.
- **Contador de Programa (PC):** registrador síncrono de 32 bits com reset, descrito internamente no topo do processador; sua lógica de atualização foi apresentada e será integrada à simulação final.

Todos esses componentes foram simulados individualmente no ModelSim, e os resultados documentados por meio de capturas de tela (waveforms) atestam o funcionamento conforme a especificação RISC-V.

A escolha da arquitetura RISC-V e da linguagem SystemVerilog mostrou-se adequada, permitindo uma descrição clara e modular dos componentes. As ferramentas de simulação empregadas (ModelSim) foram suficientes para a validação funcional. Espera-se que, ao final da próxima etapa, o processador esteja completamente implementado e pronto para ser sintetizado em um kit FPGA, executando programas de teste em hardware real.

5.0.1 Dificuldades Encontradas

Durante o desenvolvimento do projeto, algumas dificuldades técnicas merecem destaque, pois exigiram atenção especial e aprofundamento no entendimento da linguagem SystemVerilog e da arquitetura RISC-V.

A principal dificuldade esteve relacionada à correta manipulação de tipos com e sem sinal (*signed* vs. *unsigned*) dentro da Unidade Lógica e Aritmética (ULA). Em operações como deslocamento aritmético à direita (SRA) e comparações com sinal (SLT, SLTI), é fundamental garantir que os operandos sejam tratados como números com sinal. A linguagem SystemVerilog, por padrão, considera vetores (`logic [31:0]`) como sem sinal. A solução encontrada foi utilizar conversões explícitas com `$signed()` nos operandos, como exemplificado no trecho:

```
1 SRA: aluout_o = $signed(op_1) >>> op_2[4:0];
2 SLT: aluout_o[0] = ($signed(op_1) < $signed(op_2)) ? 1'b1 : 1'b0;
```

A ausência desses *casts* resultava em comportamentos incorretos, especialmente quando operandos negativos eram envolvidos. A dificuldade foi ainda maior nas operações de multiplicação e divisão da extensão M, onde era necessário tratar corretamente os produtos de 64 bits (MUL, MULH, MULHSU, MULHU) e as divisões com sinal (DIV, REM).

Outro ponto desafiador foi a implementação das instruções que utilizam a parte superior ou inferior de um produto de 64 bits (MULH, MULHSU, MULHU). Para essas operações, foi necessário armazenar o resultado completo de 64 bits em um registrador temporário (`tmp_reg`) e, em seguida, selecionar os 32 bits apropriados (`tmp_reg[63:32]` ou `tmp_reg[31:0]`). A lógica de extensão de sinal para os operandos em MULHSU (um operando com sinal e outro sem sinal) exigiu a criação de um operando estendido com `{1'b0, op_2}` para garantir a interpretação correta.

Além disso, a lógica de *byte enable* na memória de dados para acessos a byte e meia palavra (`lb`, `lbu`, `lh`, `lhu`, `sb`, `sh`) demandou cuidado no alinhamento de endereços e na geração correta dos sinais de habilitação (`be`). A simulação desses casos com ende-

reços não alinhados (ex.: `addr_i[0]=1` para acesso a byte) foi essencial para validar a implementação.

Todas essas dificuldades foram superadas com estudo da documentação do SystemVerilog, simulações incrementais e a utilização de testbenches dedicados para cada módulo, o que permitiu isolar e corrigir os problemas antes da integração final.

Referências

PATTERSON, D. A.; HENNESSY, J. L. *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*. 1. ed. [S.l.]: Morgan Kaufmann, 2018. Citado na página 10.

RISC-V INTERNATIONAL. *RISC-V Instruction Set Manual, Volume I*. 2024. Acesso em: 04 abril 2026. Disponível em: <riscv.org>. Citado 3 vezes nas páginas 19, 20 e 26.

STALLINGS, W. *Arquitetura e Organização de Computadores*. 10. ed. São Paulo: Pearson, 2017. Citado na página 17.